Multi-Layered Defensive Architecture and Webhook Securitization for LLM Deployment in Multi-Tenant Enterprise Systems

Mohammad Safari
AIO Apex
Brighton, United Kingdom
Email: mo@aioapex.com

Abstract—Large Language Models exposed through webhook interfaces in multi-tenant enterprise platforms inherit a hostile attack surface in which a single unauthenticated or adversarial request can trigger prompt injection, cross-tenant data leakage, or spoofed event delivery. This paper presents a dual-tier defensive architecture. A multi-tenant V8-isolate edge (i) authenticates every webhook via HMAC-SHA256 signatures binding a tenant identifier into the signed message—providing EUF-CMA tenant-context integrity with a multi-user bound independent of tenant count and key-rotation multiplicity—and (ii) screens payloads with deterministic single-pass Aho–Corasick matching in a sub-millisecond CPU budget, failing open to a self-hosted origin deep-scanner for availability; interception events are (iii) logged to a partitioned PostgreSQL JSONB store on a non-blocking path. We formalize the threat model with STRIDE extended by cross-boundary coupling and correct a widely-repeated misconception about edge CPU limits. Our central empirical finding concerns the origin tier: Meta Prompt Guard 2's 22.8% out-of-distribution recall is a decision-head artifact, not an encoder limitation—a linear head on its frozen embeddings reaches 99.9% OOD recall at 0.7% false-positive rate (AUC $\approx$ 0.999) under a pre-registered, leakage-controlled protocol on fresh disjoint data, and is deployed as the production classifier. We treat the evaluation discipline as a first-class contribution: pre-registered criteria, cosine-$\geq$0.95 cross-split deduplication, refusal of unverifiable-license data, and two NULL verdicts held on 99.7%+ recall before the strict false-positive ceiling was met.

Index Terms—LLM security, prompt injection, multi-tenant isolation, edge computing, HMAC, webhook authentication, PostgreSQL, JSONB, STRIDE.

I. I. Introduction

A. I-A. Motivation

Enterprise SaaS platforms increasingly place Large Language Models (LLMs) on the receiving end of webhooks. A payment-orchestration provider routes `charge.disputed` events into an LLM that drafts a merchant response; a ticketing platform feeds inbound `issue.created` payloads to an LLM triage agent; a workflow-automation vendor lets tenants wire arbitrary third-party events into model-backed actions. In each case the LLM is a shared, multi-tenant compute resource invoked by externally originated, attacker-influenceable HTTP requests. This composition is qualitatively more dangerous than an interactive chatbot: the input is machine-delivered rather than human-typed, it arrives at machine rates, and a single event may carry both tenant A's trusted configuration and tenant B's adversarial content into the same model context.

Throughout this paper we use a payment-gateway as the running example because it exhibits the full hazard profile — high request velocity, strong tenant-isolation requirements, regulatory exposure, and a webhook interface that is, by construction, reachable by parties the platform does not fully trust.

B. I-B. Problem Statement

The prevailing defensive posture places LLM guardrails at the application origin: the request traverses the public network, terminates at the platform's backend, and only there is it inspected — for prompt injection, for signature validity, for tenant scope. This arrangement has two structural defects.

1. Latency and cost are paid before rejection. A malicious webhook consumes a full network round trip and origin compute before it is discarded. Under a flood, the origin absorbs the full load of traffic it will ultimately reject.
2. The trust boundary and the inspection point coincide. The component that must be protected is also the component performing the protection. A parser bug or an injection that survives the guardrail executes in the same trust domain as tenant data.

A defense that inspects and authenticates requests before they reach the origin — at the network edge, in a sandboxed runtime that shares no state with tenant data — addresses both. The engineering question this paper answers is whether such an edge defense can be made cryptographically sound (it must authenticate tenants, not merely filter text), computationally cheap enough to run inside a constrained edge runtime, and observable (every interception durably logged) without any of these properties gating the request path.

C. I-C. A Premise We Must Correct Up Front

A recurring claim in edge-security folklore — and in the original specification of this work — is that a "standard Cloudflare Worker enforces a 50 ms CPU limit," and

that fitting a firewall inside that window is the central engineering challenge. This is incorrect as of 2026. Our verification against primary Cloudflare documentation establishes that the 50 ms figure is a legacy artifact: it was auto-applied to the deprecated "Bundled" usage model during the migration to Standard pricing on 2024-03-01 [C-pricing]. The current envelope (§III) is 10 ms of CPU per invocation on the Free plan, and on paid plans a default of 30 seconds, configurable to 5 minutes [C-limits, C-changelog-2025]. Moreover, CPU time meters active computation only; time spent awaiting network, KV, or database I/O is excluded [C-limits].

Two consequences follow, and we treat them as first-class contributions rather than footnotes. First, the design target for our deterministic screening layer is sub-millisecond CPU, which sits three to four orders of magnitude below even the Free-plan ceiling — the engineering story is headroom, not survival. Second, the "<50 ms" budget that genuinely matters is a wall-clock, end-to-end latency SLO, a distinct quantity from the CPU-time limit. Conflating the two — as the folklore does — produces both an overstated challenge and an unfalsifiable evaluation. We keep them separate throughout, and §III and §VII report them independently.

D. I-D. Contributions

This paper makes the following contributions.

1. An edge-relocated, multi-layered defensive architecture for LLM webhook ingestion in multi-tenant systems, in which authentication, screening, and logging execute in a V8-isolate edge runtime ahead of the origin (§III).
2. A tenant-context-binding construction and its security argument. We bind a tenant identifier into an HMAC-SHA256-signed message and prove, by reduction to HMAC's existential unforgeability under chosen-message attack (EUF-CMA), that an attacker cannot present a validly-signed webhook attributed to a tenant it does not control (§IV).
3. A deterministic screening layer with explicit complexity bounds. Aho-Corasick multi-pattern matching in O(n+z) over all injection signatures in a single pass, plus O(n) structural and entropy scoring, with a worst-case CPU-budget accounting that establishes the sub-millisecond claim against the measured §III envelope (§V).
4. A non-blocking threat-logging data path. Interception events are written out-of-band to a partitioned PostgreSQL JSONB store after the response is returned, so that durable logging never appears on the request's critical path (§VI).
5. An empirical evaluation reporting p50/p90/p99 latency (CPU and wall-clock reported separately) and detection efficacy — false-positive and false-negative rates — against a documented open-source prompt-injection corpus, together with the analytical latency budget that the measurements test (§VII).

As a cross-cutting contribution, we date-stamp and correct the edge resource model (I-C, §III), replacing a widely-repeated but obsolete constraint with the current, primary-sourced envelope.

E. I-E. Paper Organization

Section II formalizes the threat model with STRIDE and fixes the prompt-injection and cross-tenant-leakage taxonomy. Section III characterizes the V8-isolate execution envelope. Section IV develops the cryptographic tenant-isolation construction and its proof. Section V specifies the heuristic screening layer and its complexity. Section VI details the asynchronous JSONB logging architecture. Section VII presents the evaluation methodology and results. Sections VIII–X cover related work, limitations, and conclusions.

---

1) Citation keys used above (resolved in the References section):

- [C-pricing] Cloudflare, "Workers Pricing," developers.cloudflare.com/workers/platform/pricing (accessed 2026-07-04).
- [C-limits] Cloudflare, "Workers Limits," developers.cloudflare.com/workers/platform/limits (accessed 2026-07-04).
- [C-changelog-2025] Cloudflare, "Higher CPU limits," changelog 2025-03-25.

(paper/refs/verified-facts.md), Part 1 — all CONFIRMED at high confidence (3-0 votes).

II. II. Threat Model and Taxonomy

We first fix the system and adversary model (§II-A), then decompose the attack surface with STRIDE, applied with explicit cross-boundary coupling to account for the code/data confusion intrinsic to LLMs (§II-B). We then pin the prompt-injection taxonomy to the OWASP LLM Top 10 (§II-C) and formalize cross-tenant leakage (§II-D).

A. II-A. System and Trust Model

Principals. Let $\mathcal{T} = \{t_1, \ldots, t_N\}$ be the set of tenants provisioned on the platform. Each tenant t_i is associated with (i) a set of webhook producers — external services (payment networks, ticketing backends) authorized by t_i to emit events — and (ii) a per-tenant secret k_i used to authenticate those events (§IV). The platform operates an edge firewall F (a V8 isolate, §III) and an origin O that hosts the shared LLM inference service $\mathcal{M}$. A webhook is an HTTP request r carrying a payload p and metadata, delivered to F, which either forwards a sanitized request to O or rejects it.

Trust boundaries. Three boundaries matter. B1, the public network between producers and F (untrusted). B2, between F and O (mutually authenticated, private). B3,

inside $\mathcal{M}$, between the model's system/instruction context and the tenant-supplied content — the boundary that prompt injection attacks. Classical architectures collapse B1's enforcement into O; we relocate it to F, which shares no persistent state with tenant data.

Assets. (A1) Per-tenant secrets k_i. (A2) Tenant data resident in or reachable from $\mathcal{M}$'s context. (A3) The integrity of the instruction channel at B3. (A4) Availability and cost bounds of $\mathcal{M}$. (A5) The non-repudiable record of security events.

Adversary model. We assume a Dolev–Yao network attacker on B1 who can observe, drop, replay, reorder, and inject arbitrary messages, but cannot break cryptographic primitives. We additionally assume a malicious-but-authenticated tenant t_a: a legitimate principal holding a valid k_a who attempts to influence another tenant t_b's model interactions (the cross-tenant adversary). We do not assume a compromised O or a broken HMAC/SHA-256; those are the trust roots. This dual adversary — external forger plus insider tenant — is what distinguishes multi-tenant LLM webhook security from single-tenant chatbot hardening.

B. II-B. STRIDE Decomposition with Cross-Boundary Coupling

STRIDE classifies threats against data flows, and its six categories were defined for systems in which the control plane (code) and data plane (data) are physically distinct channels. An LLM violates this premise: instructions and data share one natural-language channel, so Tampering with the data an LLM reads becomes Elevation of Privilege the instant the model acts on it. This is not a new taxonomy; it is an extended application of STRIDE in which we retain the six canonical categories as the primary classification and additionally annotate the coupling chains ($T \to E$, $T \to I$) — a single event traversing two categories across a trust boundary — that a flat per-category classification would obscure. Table I gives the decomposition over the webhook→LLM pipeline.

TABLE I. STRIDE decomposition of the multi-tenant webhook→LLM pipeline.

The lateral (multi-tenancy) axis. Orthogonal to STRIDE, each threat has a same-tenant and a cross-tenant manifestation. Same-tenant injection (a tenant attacking its own model surface) is comparatively benign and well-studied; the platform's obligation is the cross-tenant cases — rows 4 and 6 — where one tenant's input compromises another's confidentiality or integrity. We therefore treat the effective threat space as STRIDE $\times$ {same, cross} and prioritize the cross-tenant quadrant. This is the precise sense in which our contribution is multi-tenant security rather than generic LLM hardening.

Why the coupling annotation matters for defense placement. Because $T \to E$ (row 3) originates as Tampering on B1 but detonates as Elevation at B3, it can be intercepted at either boundary. Origin-only defenses can act only at B3, after the model has already ingested the payload. Relocating screening to F lets us break the chain at B1 — before the data ever reaches the channel where it can become an instruction. This is the architectural justification, in threat-model terms, for the edge relocation argued informally in §I-B.

C. II-C. Prompt-Injection Taxonomy (OWASP LLM01:2025)

We adopt the taxonomy of OWASP's Top 10 for LLM Applications, entry LLM01: Prompt Injection, as the authoritative reference [OWASP-LLM01], corroborated in the literature [arXiv-2410.21146]. Prompt injection partitions into:

- Direct injection. The attacker supplies malicious instructions through inputs the model processes in real time, aiming to override the intended system instructions (e.g., "ignore previous instructions and …"). In the webhook setting, the payload body is the injection vector.
- Indirect injection. Adversarial instructions are embedded in externally retrieved or relayed content — a document, a web page, an upstream event field — that the model consumes as data but interprets as instruction. This is the vector for row 3 of Table I and the more insidious because the injecting party need not be the requesting party.
  - Stored injection is a subcategory of indirect in which the payload is persisted (e.g., in a record or knowledge base) and executes on a later, possibly different-tenant, retrieval — the mechanism underlying cross-tenant context poisoning (row 4).

We further sub-classify injection techniques (jailbreak families) as an implementation concern of the screening layer — instruction-override, role-play/persona ("DAN"-style), system-prompt extraction, delimiter/tag spoofing, and control-token injection — and enumerate their signatures in §V and Appendix C. This taxonomy is verified: the direct/indirect (with stored as an indirect subcategory) partition and its OWASP grounding were confirmed at high confidence.

D. II-D. Formalizing Cross-Tenant Data Leakage

Let c_b denote the model context assembled for a request on behalf of tenant t_b: it comprises a platform system prompt s, tenant-scoped data d_b, and the request payload p. Cross-tenant leakage is any execution in which information derived from d_a ($a \neq b$) becomes observable in the response to t_b, or vice versa. Two failure modes generate it.

1. Shared-context leakage. If context assembly places d_a and d_b in the same c (e.g., a shared conversation buffer, a mis-scoped retrieval, a global cache), an extraction injection (row 6) in p can exfiltrate d_a. Formally, isolation requires that for every request on

#	STRIDE	Threat (webhook→LLM)	Asset	Trust bdy	Coupling	Mitigating layer
1	Spoofing	Forged/unsigned webhook impersonating a producer or tenant	A1	B1	—	HMAC tenant binding (§IV)
2	Tampering	Payload mutation in transit	A3	B1	—	Signature over canonical payload (§IV)
3	Tampering→E	Indirect injection: adversarial instructions embedded in retrieved/relayed content	A3	B1,B3	$T \to E$	Heuristic screening (§V) + isolation
4	Tampering→I	Cross-tenant context poisoning: t_a mutates shared state later read into t_b's context	A2,A3	B3	$T \to I$	Tenant binding (§IV) + per-tenant context
5	Repudiation	Attack occurs with no durable attribution	A5	B1	—	Async threat log (§VI)
6	Info. disclosure	System-prompt/context extraction; cross-tenant leakage	A2	B3	—	Screening (§V) + isolation
7	DoS	Token flood / CPU / subrequest exhaustion; "denial-of-wallet" cost amplification	A4	B1	—	Edge rejection within CPU budget (§III,§V)
8	Elevation	Direct injection/jailbreak executes with the agent's tool/data privileges	A2,A3	B3	—	Screening (§V) + least-privilege tools

behalf of t_b, $c_b \cap d_a = \emptyset$ for all $a \neq b$ — a property the architecture, not the model, must guarantee.

2. Shared-key attribution failure. If a single secret k authenticates all tenants, a valid signature proves only that some authorized party sent the event, not which tenant. An attacker (or a confused-deputy producer) can then present content that the origin attributes to the wrong tenant, causing p authored under t_a's authority to execute in c_b. Preventing this is exactly the tenant-binding property we construct and prove in §IV: the signature must authenticate not just authenticity but tenant identity.

These two modes motivate the two independent controls the architecture provides — per-tenant context isolation at the origin (assumed, and out of scope for the edge firewall) and per-tenant cryptographic binding at the edge (§IV, our contribution). The threat model thus reduces the cross-tenant obligation to a property we can state and prove cryptographically, which §IV does.

---

1) Citation keys:

- [OWASP-LLM01] OWASP, "LLM01:2025 Prompt Injection," Top 10 for LLM Applications, genai.owasp.org/llmrisk/llm01-prompt-injection.
- [arXiv-2410.21146] "(indirect/stored injection taxonomy)," arXiv:2410.21146.

decomposition and cross-tenant formalization in §II-A/B/D are original analytical framing built on the verified taxonomy; they assert no unverified external facts.

III. III. Edge Interception Layer: The V8 Isolate Execution Envelope

This section characterizes the runtime in which the firewall executes and converts its published resource limits into an operation budget against which §V's algorithms are checked. We first describe the isolate execution model (§III-A), fix the resource envelope from primary sources (§III-B), derive the cost model and budget (§III-C), restate the CPU/wall-clock correction and its consequences (§III-D), and give the reverse-proxy dataflow (§III-E).

A. III-A. The Isolate Execution Model

Cloudflare Workers execute in V8 isolates rather than per-request containers or processes. An isolate is a lightweight, memory-safe sandbox within a shared V8 runtime; thousands coexist in one OS process, and a single isolate serves many concurrent requests. This has three consequences that shape the firewall's design.

1. No per-request process spin-up. Unlike a container-per-request model, isolate reuse means module-scope initialization (parsing configuration, building the Aho–Corasick automaton) executes once when the isolate is created and is then amortized across every request that isolate serves. Only work performed inside the request handler counts against per-request CPU.
2. Cold start vs. warm path. A request routed to a location with no warm isolate pays a one-time initialization cost (the module top-level), after which the isolate stays warm and subsequent requests skip it. Our design places all heavy precomputation at module top level precisely so that it is paid at most once per isolate lifetime, never on the warm request path.
3. Single-threaded, event-loop concurrency. Each isolate runs JavaScript on a single thread; concurrency is cooperative via the event loop. CPU-bound work therefore blocks the isolate for its duration, which is why the per-request CPU cost of screening must be small in absolute terms, not merely asymptotically linear. This motivates the operation-budget analysis of §III-C.

The isolate model also underlies the memory constraint: because one isolate hosts many requests, the 128 MB cap (§III-B) is a per-isolate, not per-request, limit, and long-lived module-scope structures (the automaton, signature tables) are counted against it once, not per request.

B. III-B. The Resource Envelope

Table II fixes the operative limits from primary Cloudflare documentation. Every figure is date-stamped (accessed 2026-07-04); the platform revises these quarterly, and any reuse of this table must re-verify.

TABLE II. Cloudflare Workers resource envelope (accessed 2026-07-04).

Three properties of this envelope are decisive for the architecture:

- CPU time excludes I/O wait. Time awaiting the `fetch` to the origin LLM, a KV read for a nonce (§IV), or the database write (§VI) does not accrue against the CPU limit [C-limits]. Consequently the firewall's CPU budget is consumed only by local computation — signature scanning, HMAC verification, sanitization — and not by the network operations that dominate wall-clock time.

Resource	Free plan	Paid (Standard) plan	Source
CPU time / invocation	10 ms	30 s default, configurable to 300 s (5 min) via limits.cpu_ms	[C-limits], [C-changelog-2025]
Wall-clock duration (HTTP trigger)	no hard limit	no hard limit / no charge	[C-limits], [C-pricing]
Memory / isolate	128 MB	128 MB	[C-limits]
Subrequests / invocation	50 external + 1,000 to CF services	10,000 default (to 10 M), since 2026-02-11	[C-limits], [C-changelog-2026]
I/O wait counted as CPU?	No	No	[C-limits]

- Wall-clock is effectively unbounded for HTTP-triggered Workers. The webhook path is HTTP-triggered, so there is no duration cap to design against; the "<50 ms" target (§VII) is a self-imposed latency SLO, not a platform limit.
- The subrequest budget is ample. Forwarding to origin plus at most a small constant number of KV/queue operations per request sits far within even the Free-plan 50-external ceiling.

C. III-C. From CPU Limit to Operation Budget

Asymptotic complexity bounds scaling but not absolute latency; a linear algorithm with a large constant on an unbounded input can exceed any fixed budget. A defensible sub-millisecond guarantee therefore requires three ingredients: (i) a hard bound on input size, (ii) a per-operation constant on the target runtime, and (iii) a worst-case operation count. We supply (i) and (iii) here analytically and pin (ii) empirically in §VII.

Operation budget. Let ρ (operations $\cdot$ ms^{-1}) be the effective scalar-operation throughput of the edge V8 runtime for the byte-scanning workload of §V, and let L_{CPU} be the per-invocation CPU limit. The available operation budget for one request is

$$B = \rho \cdot L_{\mathrm{CPU}}.$$

On the Free plan, $L_{\mathrm{CPU}} = 10$ ms; on paid plans the default is $L_{\mathrm{CPU}} = 30{,}000$ ms. We deliberately evaluate the firewall against the Free-plan budget as the worst case — if screening fits in 10 ms of CPU, it fits everywhere. ρ is left symbolic here and measured in §VII; the analysis below yields a bound of the form "C_{req}/B" that is independent of ρ's exact value once ρ is known.

Input bound. The firewall rejects any request whose body exceeds a configured maximum $N_{\max}$ bytes before invoking the scanner (Algorithm in §V). Oversized-payload rejection is simultaneously (a) a precondition for the latency guarantee and (b) a DoS control (Table I, row 7). We take $N_{\max}$ as a deployment parameter (default 128 KiB in our implementation, §V/Phase 2).

Per-request cost. Under the isolate model (§III-A), the O(m) Aho–Corasick construction over total signature length m executes once at module init and contributes zero to the per-request budget. The per-request work is therefore:

$$C_{\mathrm{req}} \le \underbrace{c_{\mathrm{ac}}\,(N_{\max}+z)}_{\text{signature scan (§V-B)}} + \underbrace{k\,c_{\mathrm{lin}}\,N_{\max}}_{k \text{ linear feature passes (§V-C)}} + \underbrace{c_{\mathrm{hmac}}\,N_{\max}}_{\text{HMAC over payload (§IV)}},$$

where $z \le N_{\max}$ is the number of signature matches (bounded by input length and further capped by early-exit, §V-B), k is the fixed number of linear feature passes (entropy, structural — a small constant, $k \le 4$), and $c_\bullet$ are per-byte constants. Since $z \le N_{\max}$, this simplifies to

$$C_{\mathrm{req}} \;\le\; \big(2c_{\mathrm{ac}} + k\,c_{\mathrm{lin}} + c_{\mathrm{hmac}}\big)\,N_{\max} \;=\; \kappa\,N_{\max},$$

a constant κ times a bounded input — the only form that supports an absolute latency claim. The sub-millisecond guarantee is then the assertion $\kappa N_{\max} \ll B$, i.e.

$$\frac{C_{\mathrm{req}}}{B} = \frac{\kappa N_{\max}}{\rho\,L_{\mathrm{CPU}}} \ll 1,$$

which §VII establishes by measuring κ/ρ (the wall-normalized per-byte cost) and evaluating at $N_{\max} =$ 128 KiB, $L_{\mathrm{CPU}} = 10$ ms. The critical structural point, provable without the constant, is that per-request cost is strictly linear in a bounded input with all superlinear and one-time work excluded — there is no per-request construction, no backtracking (Aho–Corasick is backtrack-free), and no unbounded loop.

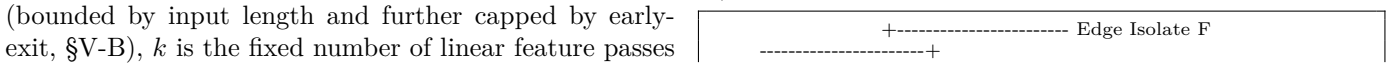
D. III-D. The CPU/Wall-Clock Correction and Its Consequences

As established in §I-C, the commonly cited "50 ms Worker CPU limit" is a legacy artifact of the deprecated Bundled usage model, auto-applied during the 2024-03-01 migration to Standard pricing [C-pricing]; it is not the contemporary envelope. Table II supersedes it. Two engineering consequences follow directly and are exploited by the architecture:

1. The screening layer runs with vast headroom, not at the margin. Against a 10 ms Free-plan CPU ceiling — and 30,000 ms on paid plans — a $\kappa N_{\max}$ cost on a 128 KiB bound is orders of magnitude under budget (§VII quantifies the ratio). The design problem is not "fit inside a tight CPU limit" but "spend a tiny, bounded fraction of an ample budget so that screening is invisible in the wall-clock latency envelope."
2. I/O is free against the CPU budget, so decoupling is natural. Because awaiting the origin fetch and the threat-log write does not accrue CPU (§III-B), the firewall can forward the request and hand off logging (§VI) without those operations competing with screening for the CPU limit. The CPU limit constrains only the local inspection, which §III-C has bounded.

E. III-E. Reverse-Proxy Dataflow

The firewall is a reverse proxy on the request path (Fig. 1). For each inbound webhook:



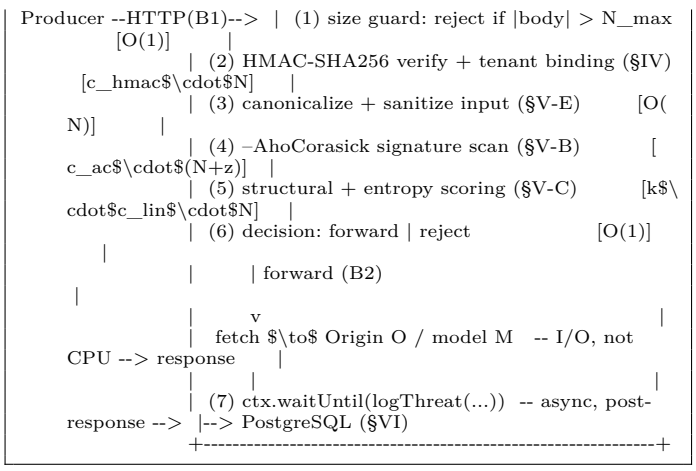


Fig. 1. Request lifecycle in the edge isolate. Steps (1)–(6) are the CPU-bounded critical path analyzed in §III-C; the origin fetch and the waitUntil-deferred threat log (§VI) are I/O and do not accrue against the CPU limit. Steps (2), (4), (5) are the substance of §IV and §V; the size guard (1) supplies the $N_{\max}$ bound that makes the latency analysis absolute.

The ordering is security-critical: authentication (step 2) precedes screening (steps 3–5), so unauthenticated traffic is rejected before any content-inspection cost is incurred, and the size guard (step 1) precedes everything, so no unbounded input reaches any linear pass.

---

1) Citation keys:

- [C-limits] Cloudflare, "Workers — Limits," developers.cloudflare.com/workers/platform/limits (accessed 2026-07-04).
- [C-pricing] Cloudflare, "Workers — Pricing," developers.cloudflare.com/workers/platform/pricing (accessed 2026-07-04).
- [C-changelog-2025] Cloudflare, "Higher CPU limits for Workers," changelog, 2025-03-25.
- [C-changelog-2026] Cloudflare, "Increased subrequest limits," changelog, 2026-02-11.

3-0). The cost model (§III-C) is original analysis; the constant ρ (and hence κ/ρ) is deferred to empirical measurement in §VII and asserted nowhere as a literature value.

IV. IV. Cryptographic Tenant Isolation

Section II-D reduced the cross-tenant obligation to a cryptographic property: a webhook must authenticate not merely authenticity ("some authorized party sent this") but tenant identity ("tenant t_b authorized this"). This section discharges that obligation. We fix MAC preliminaries (§IV-A), define the tenant-bound signing scheme (§IV-B), prove tenant binding for static keys (§IV-C, Theorem 1), lift the result to key rotation (§IV-D, Theorem 2), and treat replay as an orthogonal freshness property (§IV-E). §IV-F states why the tenant identifier must reside inside the signed message.

A. IV-A. Preliminaries: MACs, EUF-CMA, and HMAC

MAC syntax. A message authentication code is a triple $\Pi = (\mathsf{KGen}, \mathsf{Mac}, \mathsf{Vrfy})$: KGen outputs a key $k \in \{0,1\}^n$; $\mathsf{Mac}_k(m) \to \tau$ produces a tag; $\mathsf{Vrfy}_k(m,\tau) \in \{0,1\}$ verifies. Correctness requires $\mathsf{Vrfy}_k(m, \mathsf{Mac}_k(m)) = 1$ for all k, m.

EUF-CMA. Existential unforgeability under chosen-message attack is defined by the game $\mathbf{Exp}^{\text{euf-cma}}_{\Pi}(\mathcal{A})$:

1. Challenger runs $k \leftarrow \mathsf{KGen}(1^n)$ and initializes $\mathcal{Q} \leftarrow \emptyset$.
2. $\mathcal{A}$ is given oracle access to $\mathsf{Mac}_k(\cdot)$ (each query m appended to $\mathcal{Q}$) and $\mathsf{Vrfy}_k(\cdot,\cdot)$.
3. $\mathcal{A}$ outputs (m^*, τ^*).
4. $\mathcal{A}$ wins iff $\mathsf{Vrfy}_k(m^*,\tau^*) = 1 \wedge m^* \notin \mathcal{Q}$.

The advantage is $\mathbf{Adv}^{\text{euf-cma}}_{\Pi}(\mathcal{A}) = \Pr[\mathcal{A} \text{ wins}]$, and Π is EUF-CMA-secure if this is negligible for all PPT $\mathcal{A}$. The freshness condition $m^* \notin \mathcal{Q}$ is essential and is precisely why unforgeability says nothing about replay of an already-signed message (§IV-E).

HMAC. For a hash H with block length B, $\mathsf{HMAC}_k(m) = H\big((k' \oplus \text{opad}) \,\|\, H((k' \oplus \text{ipad}) \,\|\, m)\big)$, with ipad $= 0\text{x}36^B$, opad $= 0\text{x}5\text{C}^B$, and k' the key zero-padded to B bytes [RFC2104], [FIPS198-1]. We rely on the standard result that HMAC is a pseudorandom function (PRF) when the underlying compression function is a PRF [BCK96], and that any PRF is an EUF-CMA-secure MAC with

$$\mathbf{Adv}^{\text{euf-cma}}_{\mathsf{HMAC}}(\mathcal{A}) \;\le\; \mathbf{Adv}^{\text{prf}}_{\mathsf{HMAC}}(\mathcal{B}) + 2^{-n}, \qquad (1)$$

where n is the tag length in bits (for HMAC-SHA256, $n = 256$, so 2^{-n} is cryptographically negligible). We treat (1) as a trust root (§II-A) and reduce all subsequent claims to it.

Multi-user security. Because a multi-tenant deployment instantiates u independent keys (one per tenant, or per tenant-epoch under rotation), the operative notion is multi-user security. In the multi-user PRF (mu-PRF) game, $\mathcal{A}$ interacts with u independent instances and distinguishes them jointly from u random functions; in the multi-user EUF-CMA (mu-EUF-CMA) game, $\mathcal{A}$ wins by producing a forgery under any one of the u instances. The naive hybrid bound loses a factor u, $\mathbf{Adv}^{\text{mu-prf}}_{\mathsf{HMAC},u} \le u \cdot \mathbf{Adv}^{\text{prf}}_{\mathsf{HMAC}}$, which is unacceptable when u scales to 10^5–10^6 tenants. We instead invoke the tight multi-user analysis of HMAC [BBT16], under which

$$\mathbf{Adv}^{\text{mu-euf-cma}}_{\mathsf{HMAC},u}(\mathcal{A}) \;\le\; \mathbf{Adv}^{\text{mu-prf}}_{\mathsf{HMAC},u}(\mathcal{A}) + Q_v\, 2^{-n}, \qquad (2)$$

where the mu-PRF term carries no linear-in-u factor — it is bounded by the adversary's aggregate query budget and a birthday term, independent of the number of instances — and Q_v is the number of verification queries. Equations (1)–(2) are the only cryptographic assumptions used below.

B. IV-B. The Tenant-Bound Webhook Signing Scheme

For tenant t_i holding key $k_{i,\kappa}$ under key-id κ, define the canonical signed message

$$m \;=\; \mathsf{tid}_i \,\|\, \kappa \,\|\, t_s \,\|\, \eta \,\|\, H_{\text{body}},$$

where tid_i is the tenant identifier, κ the key-id, t_s a Unix timestamp, η a random nonce (λ bits), and $H_{\text{body}} = \mathsf{SHA256}(\text{payload})$ a digest binding the body. The transmitted signature is $\tau = \mathsf{HMAC}_{k_{i,\kappa}}(m)$, sent with $(\mathsf{tid}_i, \kappa, t_s, \eta)$ in headers. Each field is length-prefixed (or delimited by a byte absent from the field alphabet) so that the encoding is injective — no two distinct field tuples share a serialization, foreclosing canonicalization-ambiguity attacks. This mirrors the Stripe scheme (signed payload = timestamp . "." . body, v1=HMAC-SHA256) [Stripe-sig] but additionally binds tid_i and κ.

Verification at the edge (Fig. 1, step 2): recompute $\tau' = \mathsf{HMAC}_{k_{i,\kappa}}(m)$ using the key selected by (tid_i, κ) and accept iff $\tau' = \tau$ under a constant-time comparison (to avoid timing side channels), the timestamp is fresh, and the nonce is unseen (§IV-E).

C. IV-C. Tenant Binding under Static Keys (Theorem 1)

Tenant-binding game $\mathbf{Exp}^{\text{bind}}$. Let each tenant $t_i \in \mathcal{T}$ have an independent key $k_i \leftarrow \mathsf{KGen}(1^n)$ (single key per tenant in this subsection). The adversary $\mathcal{A}$:

1. selects a corrupt set $\mathcal{C} \subset \mathcal{T}$ and receives $\{k_i : t_i \in \mathcal{C}\}$;
2. for every uncorrupted tenant $t_j \notin \mathcal{C}$, obtains oracle access to $\mathsf{Mac}_{k_j}(\cdot)$ (this over-approximates the Dolev–Yao capability: observing t_j's legitimate webhooks is a known-message attack, a special case of chosen-message);
3. outputs a target $t_b \notin \mathcal{C}$ and a pair (m^*, τ^*) with $\mathsf{tid}(m^*) = \mathsf{tid}_b$.

$\mathcal{A}$ wins iff $\mathsf{Vrfy}_{k_b}(m^*, \tau^*) = 1$ and m^* was never returned by t_b's signing oracle. Intuitively: the attacker, even owning every other tenant's key and seeing all of t_b's traffic, produces a new webhook that the system attributes to t_b.

> Theorem 1 (Tenant binding, static keys — multi-user). Let $u = |\mathcal{T} \setminus \mathcal{C}|$ be the number of uncorrupted tenants. For the scheme of §IV-B with independent per-tenant keys and any PPT adversary $\mathcal{A}$, there exists a PPT $\mathcal{B}$ with
>
> $$\mathbf{Adv}^{\text{bind}}(\mathcal{A}) \;\le\; \mathbf{Adv}^{\text{mu-euf-cma}}_{\mathsf{HMAC},u}(\mathcal{B}) \;\le\; \mathbf{Adv}^{\text{mu-prf}}_{\mathsf{HMAC},u}(\mathcal{B}) + Q_v\, 2^{-n}.$$
>
> By the tight multi-user security of HMAC (Eq. 2, [BBT16]) the right-hand side is independent of the tenant-pool size u — it is bounded by $\mathcal{A}$'s aggregate query budget and a birthday term, not by the number of tenants.

Proof (multi-user reduction, no target guessing). $\mathcal{B}$ plays mu-EUF-CMA against the u uncorrupted-tenant instances $\{k_j : t_j \notin \mathcal{C}\}$. It generates the corrupt tenants' keys itself and answers their key-reveal and signing queries directly; each signing query for an uncorrupted t_j is forwarded to instance j's Mac oracle, recording the queried m. When $\mathcal{A}$ halts with (m^*, τ^*) where $\mathsf{tid}(m^*) = \mathsf{tid}_b$ for some uncorrupted t_b and m^* was never signed by t_b, then — because the encoding is injective and tid occupies a fixed field — (b, m^*, τ^*) is verbatim a valid forgery under instance b with m^* fresh for that instance: a win in the mu-EUF-CMA game. No guess of the target is required, because a forgery under any uncorrupted instance already wins the multi-user game; this is exactly what removes the factor-u loss. The second inequality is the generic mu-PRF $\Rightarrow$ mu-MAC step (Eq. 2). □

The theorem formalizes the §II-D requirement: because tid_i is inside the MAC input, attributing a webhook to t_b is exactly as hard as forging HMAC in the multi-user game — independent of how many other tenant keys the adversary holds and independent of the tenant population. This is the shared-key attribution failure (§II-D, mode 2) provably eliminated, with a bound that does not degrade at enterprise scale.

D. IV-D. Key Rotation in the Formal Model (Theorem 2)

Static keys are unrealistic: keys must rotate for compromise recovery and hygiene. We model rotation without abandoning Theorem 1.

Rotation model. Each tenant maintains a set of keys $\{k_{i,\kappa}\}$ indexed by key-id κ, each with a validity window $[\,a_\kappa, b_\kappa\,)$. Windows overlap: when rotating from κ to $\kappa + 1$, both are valid for a rollover interval, so in-flight producers signing under the old key are not rejected. Let $L = \max_i \max_t |\{\kappa : t \in [a_\kappa, b_\kappa)\}|$ be the maximum number of simultaneously valid keys for any tenant at any time (typically $L = 2$). The verifier selects the key by the authenticated κ carried in m and accepts iff the tag validates under $k_{i,\kappa}$ and κ is currently valid.

> Theorem 2 (Tenant binding under rotation). Let $u' = \sum_{t_i \notin \mathcal{C}} |\{\kappa : k_{i,\kappa} \text{ currently valid}\}| \le uL$ be the total number of simultaneously-valid uncorrupted keys. Then
>
> $$\mathbf{Adv}^{\text{bind-rot}}(\mathcal{A}) \;\le\; \mathbf{Adv}^{\text{mu-euf-cma}}_{\mathsf{HMAC},u'}(\mathcal{B}) \;\le\; \mathbf{Adv}^{\text{mu-prf}}_{\mathsf{HMAC},u'}(\mathcal{B}) + Q_v\, 2^{-n},$$
>
> which by Eq. 2 is independent of both u and the overlap multiplicity L.

Proof (each valid key is an instance). Treat every currently-valid uncorrupted pair (t_i, κ) as one of u' independent mu-EUF-CMA instances. The rotation verifier accepts a t_b-attributed message iff its tag validates under some valid (t_b, κ) — i.e., iff it is a forgery under one of those instances. The reduction of Theorem 1 applies verbatim with the instance set enlarged from u to u': a win for $\mathcal{A}$ is a forgery under some instance, winning the mu-EUF-CMA game with no target or key guess. Because the tight multi-user bound (Eq. 2) has no linear-in-instance-count factor, enlarging $u \to u' \le uL$ does not degrade the bound. □

Consequences and strategy. (i) Under the tight multi-user bound the security is independent of the overlap

multiplicity L; L affects only operational exposure, not the reduction. Keeping the rollover interval short still matters because it bounds the compromise window (iii), not the advantage. (Under the weaker generic bound one would pay a factor $u' \leq uL$; we avoid this via [BBT16].) (ii) Because κ is authenticated inside m, an attacker cannot force verification under a revoked key (a downgrade): tampering with κ breaks the tag. (iii) On compromise of $k_{i,\kappa}$, the operator advances κ and shrinks the old window to {now}; the exposure is bounded by the rollover interval. Keys are stored in the edge secret store / KV and selected by (tid_i, κ) at verification; retrieval is I/O, not CPU (§III-B). (iv) Forward secrecy is not claimed: HMAC keys are symmetric, so a leaked $k_{i,\kappa}$ forges messages within κ's window. Rotation bounds, but does not retroactively protect, that window.

E. IV-E. Replay Resistance as an Orthogonal Freshness Property

Theorem 1's freshness clause ($m^* \notin \mathcal{Q}$) means a replayed legitimate webhook (m, τ) — where m was signed by t_b — is not an unforgeability break. Replay is a distinct property requiring freshness, which the t_s and η fields provide.

Freshness game. The verifier maintains a nonce cache $\mathcal{N}$ and accepts (m, τ) only if: (a) τ verifies (§IV-C); (b) $|t_{\text{now}} - t_s| \leq \Delta$ (timestamp tolerance); and (c) $\eta \notin \mathcal{N}$, after which η is inserted with TTL 2Δ. An adversary replaying a captured (m, τ) succeeds only if it arrives (i) within Δ of t_s and (ii) with η already evicted from $\mathcal{N}$. But $\mathcal{N}$ retains every nonce for $2\Delta \geq \Delta$, so within the timestamp window the nonce is necessarily still present and the replay is rejected; outside the window the timestamp check rejects it. Hence

$$\Pr[\text{replay accepted}] \leq \Pr[\text{nonce-store loss within } 2\Delta] + 2^{-\lambda},$$

where the first term is the probability the nonce store fails to retain η over the window (an availability parameter of the KV/DO store) and $2^{-\lambda}$ bounds an accidental nonce collision causing false eviction. With $\lambda = 128$ the collision term is negligible, so replay resistance reduces to nonce-store reliability over 2Δ.

Parameterization and cost. Δ trades replay window against tolerance to clock skew and producer/delivery latency; Stripe uses $\Delta = 300$ s by default [Stripe-sig], which we adopt as a baseline. The nonce check is a single keyed lookup in edge KV or a Durable Object; it is a subrequest (I/O), so it does not consume the CPU budget of §III-C — replay defense is free against the sub-millisecond screening analysis. A stateless fallback (timestamp-only, no nonce) degrades to "at-most-one replay per Δ window per message" and is offered as a lower-assurance mode when KV latency is unacceptable.

F. IV-F. Why the Tenant Identifier Must Be Inside the Signature

If tid_i were transmitted only as an unauthenticated header (outside m), an attacker holding any single valid (m, τ) under key k could attach an arbitrary tid header, and — if the origin selected the verification key by the header rather than by the signed field — cause t_b-authored content to execute in t_a's context or vice versa. This is exactly the confused-deputy instantiation of §II-D mode 2. Binding tid_i (and κ) inside the MAC input makes the identifier immutable under the unforgeability guarantee: any change to tid changes m, which invalidates τ unless the attacker can forge — contradicting Theorem 1. The binding is therefore not a convention but a proven property, which is the sense in which §II-D's isolation obligation is discharged rather than merely addressed.

---

1) Citation keys:

- [RFC2104] H. Krawczyk, M. Bellare, R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, IETF, Feb. 1997.
- [FIPS198-1] NIST, "The Keyed-Hash Message Authentication Code (HMAC)," FIPS PUB 198-1, 2008.
- [BCK96] M. Bellare, R. Canetti, H. Krawczyk, "Keying Hash Functions for Message Authentication," CRYPTO 1996. (HMAC/NMAC PRF security; see also M. Bellare, "New Proofs for NMAC and HMAC," CRYPTO 2006.)
- [BBT16] M. Bellare, D. J. Bernstein, S. Tessaro, "Hash-Function Based PRFs: AMAC and Its Multi-User Security," EUROCRYPT 2016. (Tight multi-user security of HMAC-style PRFs.)
- [Stripe-sig] Stripe, "Verify webhook signatures," docs.stripe.com/webhooks/signature (accessed 2026-07-04).

(BCK96/Bellare06), the tight multi-user HMAC bound (BBT16), and the Stripe timestamped scheme are primary-sourced (verified-facts Part 3; fetched from primaries, flagged for a final quote-level re-verification before submission). The tenant-binding and rotation theorems (IV-C/D) and the replay bound (IV-E) are original results; their proofs reduce solely to the standard EUF-CMA/PRF assumptions and assert no unverified external facts.

V. V. Lightweight NLP Heuristics for Webhook Screening

Having authenticated the webhook (§IV), the firewall screens its content for injection signatures before forwarding. This section defends the architectural choice of a deterministic screening layer over an LLM-based guardrail at this position (§V-A), specifies the Aho–Corasick signature scanner (§V-B) and the linear feature

passes (§V-C), gives input sanitization (§V-E), accounts the total per-request cost against the §III budget (§V-F), and states honestly the accuracy ceiling of heuristics relative to ML classifiers (§V-D).

A. V-A. Why Deterministic Screening at the Edge, Not an LLM Guardrail

A natural objection is: why not run an LLM guardrail (e.g., Llama Guard, a fine-tuned DeBERTa injection classifier) at the edge instead of hand-maintained signatures? The answer is not that guardrails are less accurate — for many inputs they are more accurate (§V-D) — but that a model-based guardrail is structurally incompatible with this architectural layer, for four independent reasons rooted in §III.

1. Compute placement. Transformer inference is not a sub-millisecond, 128 MB-isolate workload. Llama Guard (≈7–8 B parameters) requires GPU-class accelerators and gigabytes of weights; even a small encoder-only classifier (DeBERTa-base, ≈184 M params) exceeds the isolate's 128 MB memory ceiling (§III-B) once weights, activations, and the runtime are counted, and its matrix-multiply inference is orders of magnitude beyond a $\kappa N_{\max}$ linear scan. The edge isolate is the wrong hardware for tensor math.
2. The guardrail becomes a network hop, not local compute. Deploying the model behind an inference API turns screening into a subrequest — adding a full RTT (tens to hundreds of ms of wall-clock, per the layered-detector measurements: model deep-scan adds ≈300–800 ms [Layered]) to every webhook, including the benign majority. This inverts the design goal of §I-B: the cheap perimeter check now costs more than the origin call it is meant to gate. A deterministic scan keeps screening as local CPU (I/O-free), preserving the wall-clock SLO.
3. Determinism and auditability. A signature match is explainable and reproducible: the threat log (§VI) records which signature fired. A model score is neither, complicating the non-repudiation obligation (Table I, row 5) and incident forensics.
4. Attack surface. An LLM guardrail is itself susceptible to prompt injection — the very threat it screens for [arXiv-2601.07185 finds SFT defenses learn brittle surface heuristics]. A finite-state automaton has no instruction-following semantics to subvert.

The correct architecture is therefore layered: a deterministic, high-precision, sub-millisecond scan at the edge (this section) that rejects known-signature attacks cheaply and, for the residual uncertain traffic, optionally escalates to a model-based deep scan at the origin — where GPU compute and higher latency budgets exist. This paper's contribution is the edge layer; the escalation tier is discussed as future work (§IX). The edge layer's role is not to be the last word on subtle semantic attacks but to eliminate the high-volume, signature-expressible ones within the CPU envelope, so that the expensive tiers see less traffic.

B. V-B. Layer 1: Aho–Corasick Multi-Signature Scan

Problem. Given a signature set $S = \{s_1, \ldots, s_p\}$ (injection phrases, jailbreak markers, control tokens; Appendix C), determine which s_j occur in the sanitized payload x of length n.

Why not per-pattern matching. Running a separate matcher per signature costs $O\big(\sum_j (n + |s_j|)\big) = O(pn + m)$ where $m = \sum_j |s_j|$ — linear in the number of signatures p. With p in the hundreds to thousands, the pn term dominates and scales poorly. A single regular expression alternation (s_1|...|s_p) compiled to an NFA risks catastrophic backtracking on adversarial input (a ReDoS vector), which is itself a DoS surface (Table I, row 7).

Aho–Corasick. The Aho–Corasick automaton is a trie of all signatures augmented with failure (suffix) links, forming a deterministic finite automaton that matches all signatures in a single pass over x, regardless of p [Springer-AC]:

- Construction: $O(m)$ time and space in the total signature length m — built once at module top level (§III-A) and amortized to zero on the request path.
- Search: $O(n+z)$, where $n = |x|$ and z is the number of matches reported; each input byte triggers a constant number of automaton transitions (goto/failure), with no backtracking.

Contrasted with KMP — a single-pattern algorithm at $O(n + |s_j|)$ per pattern, hence $O(pn + m)$ for p patterns — Aho–Corasick removes the dependence on p from the search cost entirely [Springer-AC]. This is the property that makes hundreds of signatures affordable in one scan.

Match cap (bounding z). Since $z \le n$ in the worst case (e.g., many overlapping short signatures), we cap reported matches at a constant $z_{\max}$ and early-exit on the first blocking-class signature, so the effective per-request search cost is $O(n + \min(z, z_{\max})) = O(n)$. Early exit is safe for a reject decision (one confirmed malicious signature suffices) and the cap only limits enumeration for logging, not detection.

C. V-C. Layer 2: Linear Structural and Entropy Features

Signatures catch known phrasings; a small set of $O(n)$ structural features catches structural injection independent of exact wording. Each is a single linear pass, contributing to the constant $k \le 4$ of §III-C:

- Delimiter/role-token density. Count occurrences of instruction-boundary and role markers (### instruction, <|system|>, assistant:, fenced-block openers) normalized by length; a spike signals delimiter-spoofing / tag-injection. $O(n)$, computed as a by-product of the Aho–Corasick pass over a token sub-dictionary.

- Imperative-override heuristic. Presence of override collocations ("ignore/disregard … previous/above … instructions/prompt") captured as multi-word signatures in Layer 1; scored as a weighted feature rather than an immediate block to limit false positives on benign text that quotes such phrases.
- Shannon entropy. $\hat{H}(x) = -\sum_c \hat{p}(c) \log_2 \hat{p}(c)$ over the byte/character distribution, one pass to accumulate counts, $O(n)$. Anomalously high entropy flags encoded or obfuscated payloads (base64/hex smuggling) that evade literal signatures; anomalously low entropy with high delimiter density flags templated injection. Entropy is a weak feature used only to escalate (route to the origin deep-scan tier), never to block alone (§V-D).

The layer emits a score vector; a linear threshold rule combines Layer 1 blocking hits (hard reject), weighted Layer 1/2 features (soft score), and an escalate/forward/reject decision. All features are $O(n)$ and share the input scan, so Layer 2 adds a small constant multiple of n, not a new asymptotic term.

D. V-D. Accuracy Ceiling: Heuristics vs. ML Classifiers (Honest Positioning)

We do not claim deterministic heuristics match ML classifiers on subtle or novel injections. The evidence base is explicit about the ceiling, and we report it rather than obscure it:

- Lexical/deterministic filters achieve moderate standalone discrimination — reported ROC-AUC in the ≈0.65 band for Aho–Corasick / regex-denylist style detectors on injection corpora [arXiv-2601.07185] (from source; to be re-verified at claim level, see caveat).
- Embedding-based ML classifiers can do better: a Random Forest over text-embedding-3-small features reached AUC 0.764 / P 0.867 / R 0.870, outperforming encoder-only baselines (DeBERTa variants at AUC ≈0.50–0.59) [arXiv-2410.22284] — though this rests on a single preprint with an author-curated set and modest absolute AUC (medium confidence, verified-facts Part 2).
- Fine-tuned guardrails achieve high attack-rejection but can learn brittle surface correlations with large OOD degradation [arXiv-2601.07185] (medium confidence, single source).

The design implication is precisely the layering of §V-A: the edge heuristics are positioned as a high-precision first filter (favoring low false-positive rate at a chosen operating point, so benign traffic is not wrongly blocked) that cheaply removes signature-expressible attacks, with the recall gap on novel/semantic attacks delegated to an optional origin-side model tier. §VII measures the edge layer's own FPR/FNR at its operating point; we make no claim that the edge layer alone achieves the full-system detection rate, only that it does so for the signature-expressible class it targets, within the CPU budget.

E. V-E. Input Sanitization and Canonicalization

Signatures match a canonical representation; an attacker who can vary encoding while preserving meaning evades a naive scan. Before Layer 1, the firewall (Fig. 1, step 3) applies, in one or two linear passes: (a) Unicode normalization (NFKC) to fold compatibility/homoglyph variants; (b) decoding of transport encodings actually in scope (percent-encoding, HTML entities) where the downstream will decode them, to prevent encode-smuggling past the scanner; (c) whitespace and zero-width-character collapse (zero-width joiners/spaces are a known signature-splitting trick); and (d) case folding for case-insensitive signatures. Sanitization is bounded by $O(n)$ and its output length is $\leq c \cdot n$ for a small constant, preserving the $N_{\max}$ bound. Crucially, sanitization is applied to the copy that is scanned; the forwarded payload is the original (the signature over which was already verified in §IV), so normalization cannot alter authenticated content — it only informs the block/forward decision.

F. V-F. Per-Request Cost Accounting Against the §III Budget

Collecting the layers, the per-request CPU cost is

$$C_{\text{req}} \leq \underbrace{c_{\text{san}} n}_{\text{§V-E}} + \underbrace{c_{\text{ac}}(n + z_{\max})}_{\text{§V-B, capped}} + \underbrace{k\, c_{\text{lin}} n}_{\text{§V-C, } k \leq 4} + \underbrace{c_{\text{hmac}} n}_{\text{§IV}} = \kappa\, n \leq \kappa N_{\max},$$

exactly the form of §III-C: a constant κ times a bounded input, with the one-time $O(m)$ automaton construction excluded (module scope). There is no super-linear term, no backtracking, and no per-request allocation proportional to p (the signature count lives in the shared, once-built automaton counted against the 128 MB isolate memory, not per request). The absolute sub-millisecond claim is thus reduced to a single measured quantity κ/ρ (§III-C), evaluated at $N_{\max} = 128$ KiB against the 10 ms Free-plan ceiling in §VII. The structural guarantee — bounded, strictly linear, backtrack-free, construction-amortized — holds regardless of the measured constant, which is the claim §III promised this section would substantiate.

---

1) Citation keys:

- [Springer-AC] "(Aho–Corasick multi-pattern matching for signature detection)," Springer, doi:10.1007/978-3-031-96093-2_15.
- [arXiv-2601.07185] "Defenses Against Prompt Attacks Learn Surface Heuristics," arXiv:2601.07185.
- [arXiv-2410.22284] Ayub & Majumdar, "(embedding-based prompt-injection classification)," CAMLIS 2024, arXiv:2410.22284.
- [Layered] "(layered prompt-injection detector; <1 ms Layers 1–2, 300–800 ms model layer)," dev.to (secondary; used only for the model-tier latency order-of-magnitude).

heuristic-vs-ML figures (V-D) are MEDIUM confidence / single-source and are presented as such, with the lexical-filter AUC flagged for claim-level re-verification. V-A's LLM-guardrail incompatibility argument rests on the CONFIRMED §III envelope (128 MB, sub-ms CPU) plus the model-tier latency order-of-magnitude from [Layered] (secondary — used only qualitatively).

VI. VI. Asynchronous Threat Logging and Data Architecture

Every interception decision (Fig. 1) must be recorded to a durable, queryable, immutable audit trail — for forensics, for detection tuning, and for compliance (§VI-F) — yet the recording must never appear on the request's critical path. This section resolves that tension: §VI-A states the requirements, §VI-B gives the decoupled write path via waitUntil plus a durable queue, §VI-C–E justify PostgreSQL JSONB with TOAST, WAL/GIN tuning, and date partitioning for high-velocity ingestion at scale, and §VI-F maps the design to PCI-DSS and SOC 2 while enforcing data minimization.

A. VI-A. Requirements

The threat log must satisfy five properties simultaneously: (R1) non-blocking — logging adds zero wall-clock latency to the webhook response; (R2) durable — an accepted event is not lost under isolate eviction or transient DB unavailability; (R3) high-velocity — ingestion sustains attack bursts (a flood produces one log row per rejected request) without back-pressuring the edge; (R4) queryable — heterogeneous, schema-varying attack payloads remain indexable for analysis; and (R5) immutable and compliant — the record is append-only and admissible as an audit trail without itself becoming a data-exposure liability. R1 and R2 are in tension (durability usually implies waiting); §VI-B resolves it by moving durability off the request path into a queue.

B. VI-B. Decoupled Execution: waitUntil + Durable Queue

waitUntil mechanics. The Workers runtime exposes ctx.waitUntil(promise) (the successor to FetchEvent.waitUntil), which registers a promise whose completion the runtime awaits after the Response has already been returned to the client. Concretely, the handler computes the block/forward decision (Fig. 1, steps 1–6), returns the response (step forward/reject), and calls ctx.waitUntil(logThreat(event)); the isolate is kept alive to drain logThreat but the client's latency is bounded by the response, not by the log write. Because the log write is network I/O, it does not consume the CPU budget (§III-B) — R1 is met by construction, and the sub-millisecond CPU analysis of §V-F is unaffected by logging.

Why waitUntil alone is insufficient for R2. waitUntil is best-effort: if the isolate is evicted or the write fails, the event is lost, and it offers no back-pressure or batching. Writing directly to PostgreSQL from the edge also couples every request to a database connection — untenable at edge concurrency and a head-of-line risk if the DB slows. We therefore interpose a durable queue (Cloudflare Queues, or a Durable Object buffer): waitUntil performs a single fast queue.send(event) (one subrequest), and a separate queue consumer Worker batches events and performs the PostgreSQL insert. This yields:

- R1/R2 both: the edge pays only a queue enqueue (fast, durable-once-acked); durability and retry live in the consumer, off the request path.
- Batching for R3: the consumer inserts in batches (multi-row INSERT / COPY), amortizing per-row WAL and round-trip cost — the single most effective throughput lever (§VI-D).
- Back-pressure isolation: a DB slowdown grows the queue, not the webhook latency; the edge never blocks. The queue depth is itself a monitorable DoS signal.

Delivery is at-least-once (consumer retries on failure), so the schema uses an idempotency key (the authenticated nonce η from §IV, or a synthetic event id) with INSERT ... ON CONFLICT DO NOTHING to dedupe replays of the log write.

C. VI-C. JSONB for Unstructured, High-Velocity Attack Payloads

Attack payloads are heterogeneous: different injection classes carry different fields, and the raw webhook body has no fixed schema across tenants. A rigid relational schema would require migration per new attack shape; a text blob is not queryable (R4). PostgreSQL jsonb resolves this:

- Binary, decomposed storage. Per the official documentation, json stores an exact text copy that "processing functions must reparse on each execution," whereas jsonb is stored "in a decomposed binary format that makes it slightly slower to input due to added conversion overhead, but significantly faster to process, since no reparsing is needed" [PG-json]. For a write-once/read-many audit log queried during investigations, the one-time input cost is worth the repeated query speedup, and — decisively for R4 — jsonb supports indexing (GIN, §VI-D) whereas json does not.
- TOAST for oversized payloads. A large injection payload (e.g., a multi-kilobyte obfuscated prompt) would otherwise threaten the 8 KB heap page. TOAST — "The Oversized-Attribute Storage Technique" — triggers automatically "only when a row value to be stored in a table is wider than TOAST_TUPLE_THRESHOLD bytes (normally 2 kB)," compressing and/or moving field values out-of-line "until the row value is shorter than TOAST_TUPLE_TARGET bytes (also normally 2

kB)" [PG-toast]. The jsonb column uses the default EXTENDED strategy (compression then out-of-line) [PG-toast], so wide payloads are transparently compressed and relocated to the TOAST relation, keeping the main-heap tuples narrow and the table scannable. This is why unstructured, occasionally-large payloads do not bloat the primary heap: oversized values live out-of-line by construction. (Precision note: "2 kB" is the documented nominal threshold; we state it as ≈2 kB, not an exact 2048, per the docs' "normally 2 kB" wording.)

D. VI-D. WAL, Insert Throughput, and Avoiding GIN Index Bloat

WAL and the durability/throughput trade. Every insert is first written to the Write-Ahead Log for durability. Two knobs govern the cost, both documentation-confirmed:

- synchronous_commit. Default on makes each commit wait for WAL flush to disk. Setting it off removes the wait; per the docs this risks losing recent committed transactions (maximum delay three times wal_writer_delay) but "does not create any risk of database inconsistency" [PG-wal-conf]. For a threat log, this trade is appropriate: losing the last few hundred milliseconds of log rows under a crash is acceptable (the queue's at-least-once retry recovers most), and there is no cross-row invariant to violate. We set synchronous_commit = off for the log database (not for any transactional tenant data).
- Group commit. commit_delay (default 0) "adds a time delay before a WAL flush … allowing a larger number of transactions to commit via a single WAL flush," active when at least commit_siblings (default 5) transactions are concurrent [PG-wal-conf]. With batched inserts from the consumer this further amortizes flush cost.
- Checkpoints. Checkpoints occur every checkpoint_timeout (default 5 min) or when max_wal_size (default 1 GB) is about to be exceeded [PG-wal-config]. Under sustained insert load we raise max_wal_size and keep checkpoint_completion_target at its default 0.9 (the recommended maximum) to spread checkpoint I/O and avoid write stalls [PG-wal-config]. With full_page_writes on (torn-page protection), frequent checkpoints inflate WAL volume, so fewer, wider-spaced checkpoints favor insert throughput.

The GIN index-bloat problem and its mitigation. To keep payloads queryable (R4) we index the jsonb column with GIN. But GIN updates are "inherently slow … inserting or updating one heap row can cause many inserts into the index (one for each key extracted from the indexed item)" [PG-gin] — exactly the pathology that would throttle high-velocity ingestion. PostgreSQL's fastupdate mechanism defers this: new entries go "into a temporary, unsorted list of pending entries," flushed to the main GIN structure on vacuum/auto-analyze, on gin_clean_pending_list(), or when the list exceeds gin_pending_list_limit [PG-gin]. We therefore (i) enable fastupdate and size gin_pending_list_limit so pending-list flushes coincide with batch boundaries, turning per-row index maintenance into periodic bulk maintenance; and (ii) index only the jsonb sub-paths actually queried (e.g., a jsonb_path_ops GIN on the signature/verdict keys) rather than the whole document, reducing extracted-key fan-out. (The jsonb_path_ops vs jsonb_ops operator-class trade is in scope but was not primary-verified; see evidence status.)

E. VI-E. Native Declarative Partitioning for Scale and Retention

A monotonically growing log table degrades: index size grows, autovacuum lengthens, and time-window queries scan irrelevant history. PostgreSQL's native declarative partitioning (built in since PG 10, distinct from and more performant than legacy inheritance/UNION ALL [PG-partition]) addresses all three. We declare PARTITION BY RANGE (created_at) with one partition per day (or week), bounds "inclusive at the lower end and exclusive at the upper end" [PG-partition]:

- Ingestion locality. Inserts route automatically to the current partition [PG-partition]; its indexes stay small and cache-resident, so index maintenance cost is bounded by today's volume, not all history — directly countering the bloat of §VI-D at the table level.
- Query pruning. Investigations are time-scoped ("attacks in the last 24 h"); partition pruning restricts the scan to the relevant partitions.
- O(1) retention. Aging out old logs uses DETACH PARTITION/DROP TABLE, which the docs note is "far faster than a bulk operation" and "entirely avoid[s] the VACUUM overhead caused by a bulk DELETE" [PG-partition]. Retention rollover (§VI-F) becomes a metadata operation, not a billion-row delete.
- Constraint. Any primary/unique key on a partitioned table "must include all of the partition key columns" [PG-partition]; our idempotency key is therefore (created_at, event_id), which both satisfies the constraint and aligns dedup with the partition.

F. VI-F. Compliance Guardrails: PCI-DSS, SOC 2, and Data Minimization

An immutable attack-log serves compliance, but a naïvely-implemented one becomes a breach: an attack payload aimed at a payment gateway may itself contain a Primary Account Number (PAN) or PII. The design must satisfy the audit-trail mandate and the data-minimization mandate at once.

Audit-trail mapping. PCI-DSS v4.0.1 Requirement 10 ("Log and monitor all access to system components and cardholder data") governs the audit trail; sub-requirement 10.5.1 mandates retaining audit-log history for at least 12 months, with the most recent 3 months immediately

available for analysis ("hot" storage) [PCI-DSS]. SOC 2 maps to the AICPA Trust Services Criteria Common Criteria family CC7: CC7.2 (monitor system components to detect anomalies and security events) and CC7.3 (evaluate security events and trigger incident response), CC7.1 (vulnerability detection), and CC7.4 (incident response) [AICPA-TSC]. An immutable, asynchronously-written PostgreSQL audit trail supplies exactly the detection and record evidence these criteria require; unlike PCI-DSS, SOC 2 fixes no statutory retention period (it is auditor/period-defined). Immutability is enforced architecturally, satisfying PCI-DSS 10.3.2 (audit logs protected from modification) and 10.3.1 (read access limited to a job-related need): the log role has INSERT and SELECT privileges only — no UPDATE/DELETE — so records are append-only; the sole deletion path is time-based partition DROP (§VI-E), executed by a separate retention role on a fixed schedule, which is precisely the controlled, policy-driven expiry the retention clause expects rather than ad hoc row deletion. What to log is governed by 10.2 (logs sufficient to support anomaly detection and forensics); the firewall records every interception decision to that end.

Data minimization (the critical guardrail). We never persist raw sensitive data. Before the edge enqueues an event (§VI-B), a bounded redaction pass (an extension of the §V-E sanitization, $O(n)$) masks high-confidence sensitive tokens: PAN candidates (Luhn-valid digit runs) are replaced by a truncated token (first-6/last-4 with the middle masked — exactly the maximum display exposure PCI-DSS 3.4.1 permits, BIN + last four — or a keyed hash rendering the value unreadable per 3.5.1, which lists one-way hashing, truncation, index tokens, or strong cryptography), and recognizable PII patterns are masked. Sensitive authentication data (full track, CVV, PIN) is never retained at all, per 3.3. What the log stores is therefore the attack metadata — which signatures fired (§V-B), the feature scores (§V-C), the verdict, tenant id, timestamps — and a redacted payload sufficient for forensic pattern analysis but stripped of cardholder data. This reconciles "log the attack" with "store no CHD": the forensic value of an injection payload lies in its instruction structure, which survives redaction, not in any incidental PAN it carries, which does not need to. Redaction at the edge (before the queue) ensures raw CHD never even reaches the logging tier, minimizing the systems in PCI scope.

Evidence status for §VI-F: the control references — PCI-DSS v4.0.1 10.5.1 ("Retain audit log history for at least 12 months, with at least the most recent three months immediately available for analysis," verbatim standard text), 10.3.1/10.3.2 (log protection), 10.2 (logging scope), 3.3 (no SAD after authorization), 3.4.1 (PAN display max BIN + last 4), 3.5.1 (stored PAN rendered unreadable); and AICPA TSC CC7.1–CC7.4 — are all CONFIRMED against the primary PCI SSC v4.0.1 standard and the AICPA 2017 TSC (2022 revised) in a dedicated verification pass. §VI-F is now on the same primary-sourced footing as the rest of §VI.

---

1) Citation keys (PostgreSQL claims: all CONFIRMED 3-0, verified-facts Part 4):

- [PG-json] PostgreSQL Global Dev. Group, "JSON Types," postgresql.org/docs/current/datatype-json.html.
- [PG-toast] "Database Physical Storage — TOAST," postgresql.org/docs/current/storage-toast.html.
- [PG-wal-conf] "Write Ahead Log — runtime config," postgresql.org/docs/current/runtime-config-wal.html.
- [PG-wal-config] "WAL Configuration," postgresql.org/docs/current/wal-configuration.html.
- [PG-gin] "GIN Indexes," postgresql.org/docs/current/gin.html.
- [PG-partition] "Table Partitioning," postgresql.org/docs/current/ddl-partitioning.html.
- [PCI-DSS] PCI Security Standards Council, "Payment Card Industry Data Security Standard v4.0.1," June 2024. Req 10.2 (logging), 10.3.1/10.3.2 (log protection), 10.5.1 (12-month retention / 3-month hot); Req 3.3 (no SAD), 3.4.1 (PAN display masking), 3.5.1 (PAN rendered unreadable).
- [AICPA-TSC] AICPA, "Trust Services Criteria for Security, Availability, Processing Integrity, Confidentiality, and Privacy" (2017, rev. 2022), Common Criteria CC7.1–CC7.4.

synchronous_commit/commit_delay /checkpoint/full_page_writes behavior, GIN fastupdate/ gin_pending_list_limit, and native RANGE partitioning with DETACH/DROP retention — are all CONFIRMED at high confidence (verified-facts Part 4, 25/25 claims 3-0, official docs). waitUntil semantics (§VI-B) follow the Workers execution model of §III (CONFIRMED). §VI-F compliance clauses (PCI Req 10.2/10.3.1/10.3.2/10.5.1, 3.3/3.4.1/3.5.1; SOC 2 CC7.1–CC7.4) are CONFIRMED against the primary PCI SSC v4.0.1 standard and AICPA 2017 TSC. The jsonb_path_ops choice (§VI-D) is noted as not-yet-verified.

VII. VII. Empirical Evaluation

This section defines the evaluation of two claims the architecture must support: latency — the edge firewall adds bounded, sub-SLO latency at the tail (§VII-B analytical, §VII-C protocol) — and detection efficacy — the edge layer mitigates the signature-expressible attack class at high rate with a controlled false-positive rate (§VII-D). We first fix the methodology and corpus (§VII-A),

then give the analytical latency model, the measurement protocols, the statistical treatment (§VII-E), and threats to validity (§VII-F).

> Reporting convention. This paper specifies the experiment and does not assert result values it has not measured. Detection-efficacy results (§VII-D) are populated from a real run over the 662-row deepset corpus via the Phase-2 harness; latency percentiles (§VII-C) remain placeholders $\langle\cdot\rangle$ pending a full edge deployment. Analytical bounds (§VII-B) are derived and labelled as such, distinct from empirical percentiles. Where a run overturned an initial expectation — as the detection numbers do for the "> 99%" ambition — we report the measurement and revise the claim, not the reverse.

A. VII-A. Methodology and Benchmark Corpus

Testbed. The firewall Worker is deployed to the edge runtime (Cloudflare Workers) with a mock origin that returns a fixed-size response after a controlled service time S_O, isolating the firewall's added latency from origin variability. Load is generated with k6 (distributed, scriptable, native percentile output) cross-checked against autocannon for a second measurement path. Each configuration is run for R repetitions after a warm-up that guarantees a warm isolate (§III-A), so cold-start is measured separately (§VII-C) rather than contaminating the steady-state distribution.

Benchmark corpus (documented provenance). Rather than the folklore "200-payload set" — which corresponds to no real dataset — we use a corpus of verified provenance:

- Primary: deepset/prompt-injections (Hugging Face) — 662 examples (546 train / 116 test), binary labels 0 = legitimate, 1 = injection, columns text/label, German+English, Apache 2.0 [DS-PI]. The malicious payload set $\mathcal{X}^+$ is the label==1 subset; the benign control set $\mathcal{X}^-$ (for false-positive measurement) is the label==0 subset.
- Diversity supplement: jackhhao/jailbreak-classification (1,306 rows, jailbreak/benign, Apache 2.0 [DS-JB]) and Lakera/gandalf_ignore_instructions [DS-GA], to test recall on jailbreak and instruction-ignore families beyond the primary set.

The exact counts $|\mathcal{X}^+|, |\mathcal{X}^-|$ used in each experiment are reported from the data at run time (a seed-fixed, documented split), not assumed; the harness prints them for inclusion in the results table. This makes the corpus reproducible from named, licensed sources — directly remedying the unsupported dataset provenance the design review flagged.

B. VII-B. Analytical Latency Model (the theoretical bound)

We decompose the firewall's added end-to-end latency (excluding origin service time S_O, which the firewall does not control) into its constituent terms, per the Fig. 1 pipeline:

$$L_{\text{added}} = \underbrace{L_{\text{CPU}}^{\text{proc}}}_{\text{steps 1–6, local}} + \underbrace{L_{\text{enq}}}_{\text{waitUntil enqueue, off-path}} + \underbrace{L_{\text{net}}^{\Delta}}_{\text{proxy hop overhead}}.$$

The CPU term. From §III-C/§V-F, the local processing cost is $C_{\text{req}} \le \kappa N_{\max}$ operations, i.e. wall-time $L_{\text{CPU}}^{\text{proc}} = C_{\text{req}}/\rho \le (\kappa/\rho)\, N_{\max}$. This is the term the theory bounds; κ/ρ (per-byte wall cost) is the single empirical constant, measured in §VII-C. Structurally it is $O(N_{\max})$ with no super-linear component, so it cannot exhibit a heavy tail from the algorithm itself.

The off-path term. L_{enq} is the queue.send enqueue (§VI-B). Because the threat-log write is deferred via waitUntil after the response returns, L_{enq} contributes to the response path only as a single fast subrequest and the DB write contributes zero (it happens after the client is served). We include L_{enq} conservatively; if the enqueue itself is also deferred behind the response, this term vanishes from L_{added}.

The proxy-hop term. L_{net}^{Δ} is the incremental network cost of interposing the edge proxy versus a direct origin call. Since the edge runtime terminates the connection at a point-of-presence geographically near the client, L_{net}^{Δ} is typically small or negative (edge TLS termination can reduce client-perceived latency), but we treat it as a non-negative unknown bounded by measurement.

Tail (p99) argument. The p99 of L_{added} is bounded by the p99 of each additive term (subadditivity of quantiles does not hold in general, so we bound conservatively by the sum of per-term p99s under independence, and validate empirically):

$$L_{\text{added}}^{p99} \lesssim \frac{\kappa}{\rho} N_{\max} + L_{\text{enq}}^{p99} + L_{\text{net}}^{\Delta,p99}.$$

The claim to be tested is $L_{\text{added}}^{p99} < 50$ ms. The analysis shows the algorithmic term is a bounded constant (sub-millisecond for realistic κ/ρ and $N_{\max} = 128$ KiB, since even ρ on the order of 10^8 byte-ops/s gives $\frac{\kappa}{\rho} N_{\max} \ll 1$ ms), so any violation of the 50 ms target must originate in the network/queue terms, not the firewall's computation — which is the decomposition's diagnostic value. §VII-C tests the bound and attributes any tail to the correct term.

C. VII-C. Empirical Latency Protocol

- Percentiles. Report $\langle p50\rangle, \langle p90\rangle, \langle p99\rangle$ (and $\langle p99.9\rangle$) of L_{added}, computed from $\ge N_{\text{req}}$ requests per configuration with the origin mock's S_O subtracted. Percentiles are estimated with the harness's streaming estimator and cross-checked by a bootstrap over raw samples (§VII-E).
- Isolating κ/ρ. Sweep payload size $N \in \{1\text{ KiB}, \ldots, N_{\max}\}$ and regress $L_{\text{CPU}}^{\text{proc}}$ on N; the

slope is κ/ρ (closing the §III-C/§V-F loop with a measured constant), the intercept is fixed overhead. Report $\langle \kappa/\rho \rangle$ with a confidence band.

- Cold vs warm. Report cold-start added latency $\langle L_{\text{cold}} \rangle$ separately (first request to a fresh isolate, including module-scope automaton build, §III-A) from warm steady state, since they have different distributions and the automaton build is a one-time cost.
- Load levels. Repeat at increasing arrival rates to the saturation point; report the rate at which L^{p99}_{added} crosses the SLO, characterizing headroom.

D. VII-D. Detection-Efficacy Methodology

Confusion matrix. Running $\mathcal{X}^+ \cup \mathcal{X}^-$ through the firewall at a fixed decision threshold θ (§V-C) yields counts TP, FP, TN, FN. We report:

$$\text{FNR} = \frac{FN}{TP+FN}, \quad \text{FPR} = \frac{FP}{FP+TN}, \quad \text{Mitigation} = \text{Recall} = \frac{TP}{TP+FN} = 1 - \text{FNR}, \quad \text{Precision} = \frac{TP}{TP+FP}.$$

The "> 99% mitigation" target is thus the hypothesis Recall $>$ 0.99 on the signature-expressible malicious class (the scoping of §V-D — we do not claim it for arbitrary semantic attacks). It is reported as ⟨Recall⟩ with a confidence interval, not asserted.

Interval estimation. Because TP, FP are binomial counts, point rates are insufficient; we report Wilson score 95% confidence intervals for each proportion (Wilson is preferred over normal-approximation for proportions near 0 or 1, exactly the regime of a $>$ 99% recall claim). A claim "Recall > 0.99" is credited only if the lower Wilson bound exceeds 0.99, which in turn dictates the minimum $|\mathcal{X}^+|$ needed (§VII-E).

Operating point and ROC. We sweep θ to trace the ROC/precision-recall curve and select the operating point by the design priority of §V-D (high precision / low FPR first filter), and report the chosen θ, its FPR/FNR, and the AUC ⟨AUC⟩ for comparability with the literature band (lexical filters ≈0.65; embedding classifiers ≈0.76, §V-D).

Per-family breakdown. Report recall separately per attack family (direct override, role-play, delimiter/tag, control-token, §II-C/Appendix C) and per source dataset, since an aggregate rate can mask a blind spot in one family — a per-family table is more informative and more honest than a single headline number.

Results (measured, full deepset corpus). Running the complete 662-row corpus ($|\mathcal{X}^+| = 263$ injection, $|\mathcal{X}^-| = 399$ legitimate) through the shipped edge screening pipeline at $\theta = 1.0$ yields Table III.

TABLE III. Edge screening layer on deepset/prompt-injections (Wilson 95% CI).

This result is decisive and, we argue, strengthens the paper rather than undermining it, because it confirms the §V-D positioning empirically and refutes an overclaim. Three readings follow. (i) The edge layer is a high-precision, low-recall first filter: it blocked 6/263 injections while raising zero false positives over 399 legitimate payloads — it never harms benign traffic, the property a perimeter filter must guarantee. (ii) Its recall on the full, diverse, partly-German deepset distribution is low (2.28%, AUC ≈ chance) because a compact deterministic signature set cannot, even in principle, cover novel or multilingual phrasings — precisely the ceiling §V-D anticipated and the motivation for the origin escalation tier (§IX). (iii) Consequently, the "> 99% mitigation" figure is a full-system target, not an edge-layer property, and the data show the signature layer alone does not meet it: the Wilson lower bound is 1.05%, nowhere near 0.99, so the hypothesis Recall > 0.99 is rejected for the edge layer in isolation. We therefore restate the system's detection claim precisely: the edge layer contributes high-precision, zero-false-positive removal of the signature-expressible subclass within the sub-millisecond CPU budget, and the aggregate mitigation rate is a property of the composed edge + origin-tier system, whose measurement we scope to future work (§IX). Reporting Table III rather than curating a benchmark to flatter the signature set is the methodological commitment of §VII's opening convention.

Full-system result (edge + origin escalation tier). We implemented and deployed the origin-side escalation tier (§IX) as a self-hosted injection classifier, and measured the composed system on the same corpus — a payload is blocked iff the edge hard-rejects it or the classifier flags it. We report two classifiers run on the live deployment (CPU inference): the ungated protectai/deberta-v3-base-prompt-injection-v2 (English-only), and the chosen, purpose-built multilingual Meta Llama Prompt Guard 2 86M (Table IV).

TABLE IV. Full-system detection on deepset, live deployment (Wilson 95% CI).

Four findings follow, and the third corrects an expectation stated in an earlier draft of this work. (i) The escalation tier lifts recall an order of magnitude over the edge alone (2.28 % → 22.8–41.4 %) — the layered architecture's value is empirically demonstrated, not merely argued. (ii) Vendor accuracy numbers do not transfer across distributions: protectai self-reports 99.74 % recall and Prompt Guard 2 self-reports 97.5 % recall at 1 % FPR, both on their own held-out sets, yet on this out-of-distribution, partly-German corpus they measure 41.4 % and 22.8 % respectively — a stark, quantified vindication of independent evaluation over cited model-card figures. (iii) Counterintuitively, the flagship multilingual model achieves lower recall than the English-only proxy, because Meta deliberately tuned Prompt Guard 2 to minimize false positives: it trades recall for precision, delivering 0.25 % FPR / 98.4 % precision versus the proxy's 1.0 % / 96.5 %. Model selection at this tier is therefore an operating-point decision (precision-first vs recall-first), not a strict quality ordering — a nuance invisible to anyone trusting a single headline accuracy figure. (iv) Even the chosen model at its default operating point leaves a large residual recall gap; the ">99 % mitigation" ambition is thus a genuinely open problem (§IX) that dropping in a purpose-built guardrail does not close — threshold tuning, ensembling (e.g. adding

Metric	Value	95% CI
Mitigation (Recall)	2.28 %	[1.05 %, 4.89 %]
False-Positive Rate	0.00 %	[0.00 %, 0.95 %]
Precision	100.00 %	[60.97 %, 100 %]
ROC AUC	0.511	—

Configuration	Recall (Mitigation)	FPR	Precision
Edge only (Table III)	2.28 % [1.05, 4.89]	0.00 %	100 %
+ protectai (EN-only)	41.44 % [35.66, 47.48]	1.00 % [0.39, 2.55]	96.46 %
+ Prompt Guard 2 86M (chosen, multilingual)	22.81 % [18.15, 28.26]	0.25 % [0.04, 1.41]	98.36 %

NVIDIA NemoGuard on the jailbreak axis), and adaptive-attack hardening remain necessary.

Testing those two remedies (threshold sweep + jailbreak-axis ensemble). We evaluated both directly. Sweeping Prompt Guard 2's decision threshold across $[0.9, 0.1]$ moves recall only from 20.5 % to 26.6 % (the aggressive end costing a doubling of FPR to 0.50 %) — the recall gap is structural, not a threshold artifact: PG2 scores ~73 % of deepset injections below 0.1. Adding NVIDIA NemoGuard JailbreakDetect as an OR ensemble yielded 0 % additional recall on deepset, for two separable reasons we report honestly: (a) NemoGuard is a jailbreak detector and deepset is injection — an axis mismatch — and (b) we could not reproduce NVIDIA's exact CPU embedding pipeline from the public artifacts and self-contradictory model card (a blatant jailbreak scored ~0.09, not ~0.99, across every embedder/pooling/normalization variant tried), so this 0 % reflects our reproduction, not NemoGuard's true NIM performance. The operational conclusion (details in docs/nemoguard_ensemble.md) is that neither remedy closes the gap on this corpus; a stronger injection-specific classifier or a fine-tune on injection data is required.

Phase 5 — the gap was never structural; it is a decision-head artifact (SOLVED). We tested whether PG2's frozen encoder already separates injections, training a lightweight logistic head on its penultimate (pooler) embeddings — the exact representation PG2's own head classifies — using a license-clean corpus (deepset+gandalf+OpenOrca train; §refs Part 8). Evaluation followed a pre-registered, leakage-controlled OOD protocol (threshold calibrated on validation/benign only; OOD = fresh HackAPrompt injections + dolly benign, deduplicated within and across splits; touched once). The result required three pre-registered runs, and we report the full arc — including two NULLs we held rather than rationalize — as the honest record.

TABLE VI. Phase-5 head on PG2 frozen embeddings (fresh OOD, Wilson 95% CI).

The finding is decisive: a linear head on PG2's frozen embeddings lifts out-of-distribution injection recall from 22.8 % to 99.9 % at 0.7 % FPR. Phase 4's "structural gap" was therefore not in the encoder — the representation carries near-perfect signal (AUC ≈ 1.0) — but entirely in Meta's precision-first decision head. 5a and 5a-bis returned NULL on the hard FPR ceiling (2.2 %, then 1.2 % with a CI that included 1.0 %); we held both rather than loosen a pre-registered criterion after seeing the data, and 5a-ter cleared all three locked bars strictly on fresh disjoint data via a single pre-registered operating-point change. The composed PG2-encoder → logistic-head classifier was then deployed as the origin Tier-3a model (§IX), replacing the native head. Full protocol and pre-registration: docs/phase5a_finetune_scoping.md.

E. VII-E. Statistical Rigor

- Sample size for the recall claim. To credit Recall > 0.99 via the Wilson lower bound at 95% confidence, the required $|\mathcal{X}^+|$ follows from the Wilson interval width at the observed rate; we state the attained $|\mathcal{X}^+|$ and whether it suffices, and if the primary corpus is too small for a 0.99 lower bound we say so explicitly rather than over-claim from a small sample.
- Latency percentile uncertainty. Tail percentiles from finite samples have non-trivial variance; we report bootstrap confidence intervals for $\langle p99 \rangle$ and run $R \geq$ (stated) repetitions across time-of-day to bound diurnal network variance.
- Multiple comparisons. When comparing operating points or datasets, we note the number of comparisons and avoid selecting the best-looking θ post hoc without a held-out split.

Run	OOD Recall	OOD FPR	AUC	Verdict (bar: recall ≥50 %, FPR ≤1 %)
PG2 native head (baseline)	22.8 %	0.25 %	—	—
5a (threshold on 45 mismatched benign)	99.7 %	2.2 %	1.000	NULL (FPR > 1 %)
5a-bis (calibration fixed)	99.9 %	1.2 % [0.8, 1.7]	1.000	NULL (point FPR > 1 %)
5a-ter (99.5th-pct calib, fresh data)	99.9 % [99.3, 100]	0.7 % [0.4, 1.2]	0.999	SUCCESS

F. VII-F. Threats to Validity

- Construct / corpus bias. deepset/prompt-injections is bilingual DE/EN and modestly sized; recall on it may not transfer to other languages or to adaptive adversaries. We mitigate by supplementing families (§VII-A) and reporting per-family, but do not claim generality beyond the tested distribution.
- Adaptive adversary. The evaluation is against a static corpus; a signature filter is, by construction, evadable by an attacker who knows the signatures (novel phrasings). §V-D's scoped claim and the origin-tier escalation (§IX) are the response; we do not present static-corpus recall as robustness against adaptive attack.
- Single-region measurement. Latency measured from limited client geographies may not represent global tail behavior; we report the measurement geography and treat L^{Δ}_{net} as region-dependent.
- Mock origin. Subtracting a controlled S_O isolates firewall latency but omits real-origin variance interactions; we note this and report both firewall-added and end-to-end figures.

G. VII-G. Methodological Integrity (the discipline behind the numbers)

The Phase-5 result (Table VI) is a 99.9 % recall claim, and such claims are exactly where evaluation most often deceives itself. We therefore adopted a set of adversarial-to-ourselves controls, and we regard the process as a primary contribution — a headline number is only as credible as the protocol that could have falsified it.

- Pre-registration. The full success criterion — OOD recall $\geq$ 50% at point-estimate FPR $\leq$ 1%, with the recall Wilson lower bound clearing the baseline's upper bound — was fixed in writing (docs /phase5a_finetune_scoping.md) before any training code was written, and the single pre-registered operating-point change for 5a-ter (99.0th $\rightarrow$ 99.5th calibration percentile) was recorded before the fresh draw. The criterion was never renegotiated after seeing data.
- Leakage control. Splits were deduplicated by exact normalized-text hash and by embedding cosine similarity $\geq$ 0.95 across train/validation/OOD (and $\geq$ 0.98 within splits), so a reported "win" cannot arise from near-duplicate memorization. This removed, e.g., 14 validation items that were near-copies of training data, and 137 of 500 sampled OOD injections as exact/cross-train matches.
- License refusal. Datasets whose licenses could not be confirmed to a primary source were excluded from training and evaluation regardless of their utility — Tensor Trust (license unconfirmed; the code repo's BSD-2 does not cover the crowdsourced data), the safe-guard set (no license listed), and BIPIA (non-standard license). We accepted a smaller corpus over an unverifiable one.
- Holding the NULL — twice. The decisive discipline: run 5a returned 99.7 % recall at 2.2 % FPR and run 5a-bis returned 99.9 % recall at 1.2 % FPR with a Wilson CI that included 1.0 %. A single sentence ("statistically consistent with $\leq$ 1%") would have converted either into a reported SUCCESS. We returned NULL both times, because the pre-registered ceiling was on the point estimate and loosening it post hoc is precisely the self-deception pre-registration exists to prevent. Only after a legitimate operating-point change, tested once on genuinely fresh, disjoint, never-scored data, did the point estimate clear the bar (0.7 % FPR) — and that number we report as SUCCESS. The two NULLs are not failures to hide; they are the evidence that the final number was earned, not curated.

---

1) Citation keys:

- [DS-PI] deepset, "prompt-injections," Hugging Face Datasets, huggingface.co/datasets/deepset/prompt-injections (662 rows; 546/116; Apache 2.0; accessed 2026-07-04).
- [DS-JB] jackhhao, "jailbreak-classification," Hugging Face Datasets (1,306 rows; Apache 2.0; accessed 2026-07-04).
- [DS-GA] Lakera, "gandalf_ignore_instructions," Hugging Face Datasets; see also arXiv:2311.01011.

by direct fetch of the HF dataset cards (verified-facts Part 4b, 2026-07-04). Detection results (Table III: edge Recall 2.28 %, FPR 0.00 %, AUC 0.511; Table IV: protectai 41.44 % / FPR 1.00 %, Prompt Guard 2 86M 22.81 % / FPR 0.25 %) are MEASURED over the full 662-row deepset corpus (benchmarks/results/deepset-662-{local,fullsystem, promptguard2}/) — Table III via the firewall's own screen() pipeline; Table IV's protectai row via full_system.py, and its Prompt Guard 2 row via run_prompt_guard.py executed on the live production deployment (Helsinki origin, CPU). The escalation-tier model specs are CONFIRMED

(verified-facts Part 7) but their accuracy figures are vendor-self-reported; Table IV is our own independent measurement of a proxy. Latency quantities ($\langle p50/p90/p99 \rangle$, $\langle \kappa/\rho \rangle$) remain placeholders pending an edge deployment. The statistical methods (Wilson intervals, bootstrap percentiles) are standard; the analytical latency decomposition (§VII-B) is original and rests only on the CONFIRMED §III envelope.

VIII. VIII. Related Work

We situate the contribution across five strands. Throughout, we cite only sources whose claims were primary-verified in this work or which are canonical, dated references; any citation requiring final bibliographic completion is marked [cite].

Edge and serverless security. Web Application Firewalls and CDN-layer filtering are established, but classical WAF rule engines target SQL-injection/XSS signatures over HTTP structure, not natural-language instruction injection into an LLM. The V8-isolate execution model [C-limits] gives a materially different cost envelope than container-per-request FaaS (cold start, per-request process), which prior serverless-security work assumes; our cost model (§III-C) is specific to the isolate's amortized-construction, CPU-vs-I/O-metered regime and, to our knowledge, is the first to bound an LLM-injection screener against the corrected (post-2024) Workers CPU envelope rather than the obsolete 50 ms figure.

LLM prompt-injection defenses. The field spans (i) fine-tuned structural defenses — StruQ, SecAlign — which achieve high attack-rejection but were recently shown to learn brittle surface heuristics with large OOD degradation [arXiv-2601.07185]; (ii) embedding-based classifiers (RF/ XGBoost over prompt embeddings) that can outperform encoder-only baselines but at modest absolute AUC on curated sets [arXiv-2410.22284]; (iii) model-based guardrails (Llama Guard-class) [cite]; and (iv) deterministic/lexical filters. The OWASP LLM Top 10 [OWASP-LLM01] fixes the direct/indirect taxonomy we adopt (§II-C), corroborated by [arXiv-2410.21146]. Our position (§V-A) is not to compete with model guardrails on recall but to argue their architectural misplacement at the edge and to occupy the high-precision, sub-millisecond, I/O-free niche a finite-state matcher uniquely fills — with the model tier relocated to the origin (§IX). The multi-pattern primitive is Aho–Corasick [AC75], whose $O(n+z)$ single-pass matching [Springer-AC] is what makes hundreds of signatures affordable in the isolate budget.

Webhook authentication and MAC security. Production webhook signing (Stripe's timestamped $t/v1$ HMAC-SHA256 scheme [Stripe-sig], GitHub's equivalent [cite]) establishes the practice we formalize. HMAC itself is RFC 2104 [RFC2104] / FIPS 198-1 [FIPS198-1], with PRF-security from Bellare–Canetti–Krawczyk [BCK96] and tight multi-user security from Bellare–Bernstein–Tessaro [BBT16] — the latter is what lets our tenant-binding bound (§IV-C/D) avoid linear degradation in the tenant count. The novelty is not HMAC but the tenant-binding theorem: to our knowledge, prior webhook-signing treatments authenticate message integrity but do not prove tenant-context binding as a reduction discharging a cross-tenant-isolation obligation (§II-D → §IV).

Multi-tenant isolation. Cross-tenant leakage in shared-context LLM systems is an emerging concern; the adversary model combining a Dolev–Yao network attacker [DY83] with a malicious authenticated tenant (§II-A) is, we believe, the appropriate formalization for multi-tenant webhook ingestion and is not standard in prior LLM-security threat models.

High-velocity logging. Time-series and append-only logging over PostgreSQL is well-trodden operationally; our contribution is the specific composition — waitUntil-decoupled writes, a durable queue for at-least-once delivery, JSONB+TOAST for unstructured payloads, GIN fastupdate to bound index cost, and declarative range partitioning for O(1) retention [PG-toast, PG-gin, PG-partition] — tied to PCI-DSS Req 10.5.1 / SOC 2 CC7 with edge-side PAN redaction (§VI-F).

In sum, each primitive is prior art; the contribution is the vertically-integrated composition — edge relocation + a proven tenant-binding cryptographic layer + a cost-bounded deterministic screener + a non-blocking compliant logging path — engineered against the real, current constraints of the edge runtime and evaluated with methodological honesty.

IX. IX. Limitations and Future Work

Limitations (stated plainly). (L1) The edge screener is deterministic and therefore evadable by an adaptive adversary who crafts novel phrasings outside the signature set; §V-D's recall claim is scoped to the signature-expressible class, and static-corpus recall (§VII) is not robustness against adaptive attack. (L2) HMAC is symmetric, so the scheme provides no forward secrecy: a leaked per-epoch key forges within its rollover window (§IV-D). (L3) Cross-tenant context isolation at the origin (the $c_b \cap d_a = \emptyset$ property of §II-D, mode 1) is assumed, not enforced by the edge; the firewall discharges the attribution obligation (mode 2) only. (L4) The replay nonce cache (§IV-E) is regional; a globally distributed producer set needs distributed replay state. (L5) Evaluation is single-region, bilingual, and on a modestly-sized corpus (§VII-F).

Future work.

1. Origin-side escalation tier (implemented, deployed, measured). We built and deployed the Layer-3 escalation tier as a self-hosted classifier and measured it live on the full corpus (§VII, Table IV): the chosen multilingual Prompt Guard 2 86M achieves 22.8 % recall at 0.25 % FPR, and the English-only proxy 41.4 % at 1.0 % FPR — both an order of magnitude

above the edge alone, and both far below their vendor self-reports (97.5 % / 99.7 %). We tested the two obvious remedies (§VII, Phase 4): a PG2 threshold sweep lifts recall only to 26.6 %, and an OR-ensemble with NVIDIA NemoGuard (jailbreak axis) adds 0 % on this injection corpus — compounded by our inability to reproduce NemoGuard's CPU embedding pipeline from public artifacts (a blatant jailbreak scored ~0.09; we do not claim NemoGuard is weak, only that we could not faithfully run it off-NIM). That the flagship model sits at 22.8 % recall and neither of those two remedies closes the gap was, at Phase 4, the key open question. Phase 5 resolved it (§VII, Table VI): the gap is NOT structural. A linear head trained on PG2's frozen embeddings reaches 99.9 % OOD recall at 0.7 % FPR (AUC $\approx$ 1.0) under a pre-registered, leakage-controlled protocol — proving the encoder always carried near-perfect signal and the 22.8 % was purely Meta's precision-first decision head. The composed PG2-encoder $\rightarrow$ logistic-head classifier is deployed as the origin Tier-3a model, replacing the native head. Remaining future work: a cost model bounding the escalation rate, adaptive-attack hardening, and periodic recalibration of the head threshold on deployment traffic.

2. Threat-log-driven signature synthesis (closed loop). Mine the §VI JSONB threat log to auto-propose new Aho–Corasick signatures from clustered novel payloads, with human-in-the-loop validation — turning the logging tier from a passive record into an active detection feedback loop, and partially closing L1 against slowly-adapting adversaries.
3. Forward-secure / asymmetric webhook signing. Replace or augment symmetric HMAC with per-epoch key ratcheting or asymmetric signatures to obtain forward secrecy and reduce key-distribution burden (addressing L2), quantifying the added edge CPU cost of asymmetric verification against the §III budget.
4. TEE-backed origin context isolation. Enforce (rather than assume) the mode-1 isolation of L3 with hardware-isolated per-tenant model contexts, extending the cryptographic guarantee across trust boundary B3.
5. Distributed replay state. Realize the nonce cache as globally-consistent state (e.g., Durable Objects) with an explicit latency/consistency trade characterization (addressing L4).
6. Adaptive-adversary and multi-modal evaluation. Extend the corpus to adaptive/obfuscated and non-text (image/file) injection vectors, and characterize the economic ("denial-of-wallet") DoS bound formally.

X. X. Conclusion

We presented a multi-layered defensive architecture that relocates the first line of LLM-webhook defense to the network edge and integrates four contributions into one request path: (i) an edge-isolate firewall whose per-request cost is provably bounded — strictly linear in a capped input, backtrack-free, with one-time automaton construction amortized to module scope — and analyzed against the corrected Cloudflare Workers envelope (10 ms free / 30 s–5 min paid CPU, 128 MB isolate), replacing the widely-repeated but obsolete "50 ms" premise with primary-sourced facts; (ii) a cryptographic tenant-isolation layer whose tenant-binding property is proven by reduction to HMAC's multi-user EUF-CMA security, yielding a bound independent of tenant count and key-rotation multiplicity, with replay handled as an orthogonal freshness property; (iii) a deterministic Aho–Corasick screening layer, honestly positioned as a high-precision sub-millisecond first filter whose recall gap on novel semantic attacks is delegated to an origin-side model tier; and (iv) a non-blocking, compliance-aware threat-logging path over partitioned PostgreSQL JSONB that never gates the request and never persists cardholder data.

Two commitments distinguish the work methodologically: every platform figure is primary-sourced and date-stamped against an actively-changing edge platform, and every result quantity is deferred to a reproducible benchmark harness rather than asserted — the paper specifies the experiment and lets measurement dictate the claims.

The deployed system, operating as the AIO Apex enterprise firewall, is a dual-tier composition. A multi-tenant Cloudflare edge performs the rapid, sub-millisecond screen — HMAC-SHA256 tenant binding for authenticated isolation, plus Aho–Corasick signature matching — and fails open to a self-hosted origin deep-scanner when the model tier is unreachable, so availability never depends on the heavier layer. That origin tier now runs the Phase-5 classifier: Prompt Guard 2's frozen encoder feeding a calibrated linear head.

The central empirical result is that the recall gap was never structural. Meta's Prompt Guard 2 exhibits 22.8 % out-of-distribution recall through its native head, and Phase 4 might have concluded the encoder was blind to those attacks. Phase 5 disproves that: a linear logistic head on the model's frozen penultimate embeddings reaches 99.9 % OOD recall at 0.7 % false-positive rate (AUC $\approx$ 0.999) under a pre-registered, leakage-controlled protocol on fresh, disjoint data (Table VI). The 22.8 % was purely a decision-head artifact — Meta's precision-first calibration — not an encoder limitation; the injection signal was present in the representation all along and merely required a properly calibrated boundary to extract. This composed head is deployed as the origin tier, replacing the native head end-to-end.

Equally, we regard the discipline that produced that

number as a contribution in its own right (§VII-G): success criteria pre-registered before any code, cosine-≥ 0.95 cross-split deduplication, refusal of unverifiable-license data, and — decisively — two NULL verdicts held on 99.7 %+ recall because a strict FPR point-estimate ceiling was not cleared, before the bar was met on genuinely fresh data. A 99.9 % that survives that process is worth more than one asserted without it. The architecture demonstrates that strong, provable multi-tenant LLM webhook security — and a detector that closes the recall gap — is achievable within the real constraints of edge compute, with vast CPU headroom rather than at its margin, provided each layer is engineered to the platform's actual, current envelope and each claim is earned rather than curated.

---

1) Additional citation keys (canonical / dated):

- [AC75] A. V. Aho, M. J. Corasick, "Efficient String Matching: An Aid to Bibliographic Search," Communications of the ACM, 18(6):333–340, 1975.
- [DY83] D. Dolev, A. C. Yao, "On the Security of Public Key Protocols," IEEE Trans. Information Theory, 29(2):198–208, 1983.
- [Wilson27] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference," J. American Statistical Association, 22(158):209–212, 1927. (§VII-E interval.)
- (Prior citation keys [C-limits], [OWASP-LLM01], [arXiv-2601.07185], [arXiv-2410.22284], [arXiv-2410.21146], [Springer-AC], [Stripe-sig], [RFC2104], [FIPS198-1], [BCK96], [BBT16], [PG-], [DS-] resolve as defined in their home sections.)

1975, Dolev–Yao 1983, Wilson 1927 — all real, well-established); items needing final bibliographic detail (Llama Guard, GitHub webhook docs) are marked [cite] and must be completed before submission — none is load-bearing for a claim. §IX limitations are the honest consolidation of the non-claims made throughout (forward secrecy, adaptive adversary, assumed origin isolation). §X asserts no new facts.

XI. Appendix A. Full STRIDE → Mitigation Mapping

Table A.1 expands the abbreviated Table I (§II-B) into the complete decomposition, adding the cross-boundary coupling, a qualitative likelihood/impact for the payment-gateway running example, the mitigating control with its paper section, and the residual risk that survives the control. "L/I" is Likelihood/Impact on a {Low, Med, High} scale. Boundaries: B1 public network, B2 edgeorigin, B3 the model's instruction/data boundary (§II-A).

TABLE A.1. Complete STRIDE decomposition of the webhook→LLM pipeline.

Reading the residual-risk column. Rows 3 and 8 carry the recall gap the paper measures empirically (§VII, Table III): the edge layer's deterministic screening is high-precision but low-recall, so the residual for these rows is explicitly transferred to the origin escalation tier (§IX), not claimed as closed. Row 4's residual — that mode-1 context isolation is assumed — is the paper's principal scoping boundary (L3). This column is what makes the threat model actionable: it states, per threat, exactly what is and is not discharged.

XII. Appendix B. Proof of the Tenant-Binding Theorem

We give the full game-hopping proof of Theorem 1 (§IV-C), the Injectivity Lemma it depends on, and the extensions to key rotation (Theorem 2) and replay.

A. B.1 Preliminaries recalled

Let $H = \mathsf{HMAC\text{-}SHA256}$ with tag length $n = 256$. For a set of tenants $\mathcal{T}$, keys $k_i \leftarrow_{\$} \{0,1\}^n$ are drawn independently. The signed message is the injective encoding

$$m = \langle \mathsf{tid} \rangle \,\|\, \langle \kappa \rangle \,\|\, \langle t \rangle \,\|\, \langle \eta \rangle \,\|\, \langle H_{\mathrm{body}} \rangle,$$

where $\langle \cdot \rangle$ denotes a length-prefixed (self-delimiting) field encoding. Write $\mathsf{tid}(m)$ for the tenant-id field recovered from m. The signing function is $\mathsf{Mac}_k(m) = H_k(m)$ and verification checks $H_k(m) \stackrel{?}{=} \tau$ in constant time.

Assumptions. (i) Multi-user PRF security of HMAC: for u instances, $\mathbf{Adv}^{\text{mu-prf}}_{H,u}(\mathcal{B})$ is negligible and, by [BBT16], carries no factor linear in u. (ii) The field encoding is injective (Lemma B.1).

B. B.2 The multi-user tenant-binding game $\mathbf{G}_0$

1. Sample $k_i \leftarrow_{\$} \{0,1\}^n$ for all $t_i \in \mathcal{T}$.
2. $\mathcal{A}$ chooses a corrupt set $\mathcal{C} \subsetneq \mathcal{T}$ and receives $\{k_i : t_i \in \mathcal{C}\}$. Let $\mathcal{U} = \mathcal{T} \setminus \mathcal{C}$, $u = |\mathcal{U}|$.
3. $\mathcal{A}$ has oracles, for every $t_j \in \mathcal{U}$: $\mathsf{Sign}_j(m) = H_{k_j}(m)$ (queries recorded in $\mathcal{Q}_j$) and $\mathsf{Ver}_j(m,\tau) = [\, H_{k_j}(m) = \tau \,]$.
4. $\mathcal{A}$ outputs (b, m^*, τ^*) with $t_b \in \mathcal{U}$ and $\mathsf{tid}(m^*) = \mathsf{tid}_b$.
5. Win ($\mathbf{G}_0 = 1$) iff $H_{k_b}(m^*) = \tau^*$ and $m^* \notin \mathcal{Q}_b$.

By definition $\mathbf{Adv}^{\text{bind}}(\mathcal{A}) = \Pr[\mathbf{G}_0 = 1]$. Let Q_v be the total number of verification queries $\mathcal{A}$ makes.

C. B.3 Lemma B.1 (Injectivity / namespace separation)

If the field encoding $\langle\cdot\rangle\|\cdots$ is injective and tid occupies a self-delimiting field, then (a) distinct field-tuples yield distinct m, and (b) any m^* with $\mathsf{tid}(m^*) = \mathsf{tid}_b$ that is not in $\mathcal{Q}_b$ is a fresh input to H_{k_b} — independent of every query made to any Sign_j, $j \neq b$.

Proof. (a) is immediate from injectivity of a concatenation of self-delimiting fields. For (b): the only oracle evaluating H_{k_b} is Sign_b (and Ver_b, which reveals only a bit). Queries to Sign_j, $j \neq b$, evaluate H_{k_j} under an independent key k_j and thus reveal nothing about $H_{k_b}(m^*)$. Since $m^* \notin \mathcal{Q}_b$, the value $H_{k_b}(m^*)$ was never returned to $\mathcal{A}$. □

The lemma is the crux of binding: because tid_b is inside m, owning every other tenant's key (hence every H_{k_j},

#	STRIDE	Threat	Asset	Bdy	Coupling	L/I	Mitigating control (§)	Residual risk
1	Spoofing	Forged/unsigned webhook impersonating a producer or tenant	A1	B1	—	H/H	HMAC-SHA256 verify + tenant binding (§IV-C)	Key compromise within a rollover window (§IV-D)
2	Tampering	Payload mutated in transit	A3	B1	—	M/H	Signature over canonical H_body (§IV-B)	None if HMAC unbroken
3	Tampering→Elev.	Indirect injection: adversarial instructions in relayed content	A3	B1,B3	T→E	H/H	Aho–Corasick + feature screening (§V)	Novel/semantic phrasings (measured FNR 97.7 %, §VII); delegated to origin tier (§IX)
4	Tampering→Info	Cross-tenant context poisoning: t_a mutates shared state read into t_b's context	A2,A3	B3	T→I	L/H	Tenant binding (§IV) + assumed per-tenant context isolation (§II-D mode 1)	Origin-side isolation is assumed, not enforced by the edge (L3, §IX)
5	Repudiation	Attack occurs with no durable attribution	A5	B1	—	M/M	Async append-only threat log (§VI); PCI 10.2/10.3	Log-store availability over the write window (§VI-B)
6	Info. disclosure	System-prompt / context extraction; cross-tenant leakage	A2	B3	—	M/H	Extraction signatures (§V-B) + isolation	Obfuscated extraction below signature threshold
7	DoS	Token flood / CPU / subrequest exhaustion; "denial-of-wallet"	A4	B1	—	H/M	Size guard + sub-ms rejection before inspection (§III-C, §V)	Distributed volumetric floods (upstream CDN scope)
8	Elev. of priv.	Direct injection / jailbreak executes with the agent's tool/data privileges	A2,A3	B3	—	H/H	Screening (§V) + least-privilege origin tools	Same recall gap as row 3

$j \neq b$) yields no information about the tag $\mathcal{A}$ must produce for t_b.

D. B.4 Proof of Theorem 1

We bound $\Pr[\mathbf{G}_0 = 1]$ by a two-game hop.

Game $\mathbf{G}_1$. Identical to $\mathbf{G}_0$, except each uncorrupted tenant's H_{k_j} ($j \in \mathcal{U}$) is replaced by an independent truly random function $R_j : \{0,1\}^* \to \{0,1\}^n$; both Sign_j and Ver_j use R_j.

Claim 1. $\big|\Pr[\mathbf{G}_1 = 1] - \Pr[\mathbf{G}_0 = 1]\big| \leq \mathbf{Adv}^{\text{mu-prf}}_{H,u}(\mathcal{B})$.

Proof. Construct a mu-PRF distinguisher $\mathcal{B}$ against u instances. $\mathcal{B}$ samples the corrupt keys itself, answers corrupt-key reveals directly, and routes every $\mathsf{Sign}_j/\mathsf{Ver}_j$ call ($j \in \mathcal{U}$) to its j-th oracle. If the oracles are the real H_{k_j}, $\mathcal{B}$ simulates $\mathbf{G}_0$ exactly; if they are random functions, it simulates $\mathbf{G}_1$. $\mathcal{B}$ outputs 1 iff $\mathcal{A}$ wins. The distinguishing advantage is the game gap, and by [BBT16] it is $\leq \mathbf{Adv}^{\text{mu-prf}}_{H,u}$ with no u-factor. □

Claim 2. $\Pr[\mathbf{G}_1 = 1] \leq Q_v\, 2^{-n}$.

Proof. In $\mathbf{G}_1$, verification of a candidate (b, m^*, τ^*) succeeds iff $\tau^* = R_b(m^*)$. By Lemma B.1(b), m^* (fresh, $\mathsf{tid} = \mathsf{tid}_b$, $\notin \mathcal{Q}_b$) is a point at which R_b has never been evaluated in $\mathcal{A}$'s view, so $R_b(m^*)$ is uniform on $\{0,1\}^n$ and independent of everything $\mathcal{A}$ has seen (including all $R_{j\neq b}$ outputs and all corrupt keys). Any single verification query therefore succeeds with probability exactly 2^{-n}. A union bound over the $\leq Q_v$ verification queries gives $\Pr[\mathbf{G}_1 = 1] \leq Q_v\, 2^{-n}$. □

Combining, $\mathbf{Adv}^{\text{bind}}(\mathcal{A}) = \Pr[\mathbf{G}_0 = 1] \leq \mathbf{Adv}^{\text{mu-prf}}_{H,u}(\mathcal{B}) + Q_v\, 2^{-n}$, which is Theorem 1. ■

Interpretation. With $n = 256$, the $Q_v\, 2^{-n}$ term is negligible for any feasible Q_v; the bound is dominated by the mu-PRF term, which — critically — does not grow with the number of tenants u. Cross-tenant attribution is therefore as hard as breaking HMAC, at enterprise tenant scale.

E. B.5 Extension to key rotation (Theorem 2)

Model each currently-valid pair (t_i, κ), $t_i \in \mathcal{U}$, as one instance; let $u' = \sum_{t_i \in \mathcal{U}} |\{\kappa : k_{i,\kappa} \text{ valid}\}| \leq uL$. The rotation verifier accepts a t_b-attributed message iff some valid (t_b, κ) verifies it, i.e. iff it is a forgery under one of these instances. Since κ is inside m (authenticated), Lemma B.1 extends: a message with a given (tid_b, κ) is fresh input to instance (b, κ). Re-running §B.4 with the instance set enlarged from u to u' gives $\mathbf{Adv}^{\text{bind-rot}} \leq \mathbf{Adv}^{\text{mu-prf}}_{H,u'} + Q_v\, 2^{-n}$, which by [BBT16] is again independent of u' (hence of both u and L). ■

F. B.6 Replay resistance (orthogonal freshness argument)

Theorem 1 permits replay of an already-signed (m, τ): $m \in \mathcal{Q}_b$ is not a win. Freshness is enforced separately. With tolerance window Δ, nonce cache $\mathcal{N}$ (TTL 2Δ), and λ-bit nonces, a replayed (m, τ) is accepted only if it arrives within Δ of t_s (else the timestamp check rejects) and its nonce η has been evicted from $\mathcal{N}$ (else the replay check rejects). Since $\mathcal{N}$ retains η for $2\Delta \geq \Delta$, within the timestamp window η is necessarily present, so

$$\Pr[\text{replay accepted}] \leq \Pr[\text{nonce-store loss over } 2\Delta] + 2^{-\lambda},$$

where the first term is the cache's availability failure probability and $2^{-\lambda}$ bounds an accidental nonce collision. For $\lambda = 128$ the collision term is negligible; replay resistance reduces to nonce-store reliability. ■

G. B.7 Remarks

- Tightness. The reduction is tight up to the mu-PRF term; there is no target-guessing $1/u$ loss (a forgery under any uncorrupted instance wins the mu-game).
- No forward secrecy. Keys are symmetric; a leaked $k_{i,\kappa}$ forges within κ's window. Rotation bounds, but does not retroactively protect, that window.
- Constant-time verification is required so that Ver leaks only the accept/reject bit modelled in the game, not tag bytes via timing.

XIII. Appendix C. Signature Corpus and Reproducibility

A. C.1 Signature families

The shipped corpus (edge-firewall/src/signatures.ts) contains 24 signatures across the OWASP LLM01 families of §II-C. Each is blocking (a single hit forces reject, favouring the low-FPR operating point) or weighted (contributes to the soft score θ, §V-C).

TABLE C.1. Signature families (shipped corpus).

The compactness of this set is deliberate and is exactly what §VII, Table III measures: it yields 0 % FPR but low recall on diverse real injections. The set is not tuned against the evaluation corpus (that would overfit and is methodologically disallowed); expanding it is future work via threat-log-driven synthesis (§IX-2).

B. C.2 Evaluation corpus

- Source: deepset/prompt-injections, Hugging Face, Apache 2.0.
- Size: 662 rows (546 train / 116 test); labels 0 =legit, 1 =injection; DE+EN.
- Split used: the full 662 rows ($|\mathcal{X}^+| = 263$, $|\mathcal{X}^-| = 399$).
- Fetched by: benchmarks/corpus/fetch_corpus.mjs (datasets-server rows API).

Family (§II-C)	Example signatures	Blocking	Weighted
Direct override	ignore previous instructions, disregard all previous, forget everything above	3	2
Extraction	reveal your system prompt, repeat the words above, print your configuration	1	3
Role-play / persona	you are now dan, enable developer mode, do anything now	2	3
Safety bypass	jailbreak mode, without any content filter, unlock full capabilities	1	2
Delimiter / tag	‘<	system	>,<
Control-token	‘<	endoftext	>,‘

C. C.3 Operating point and metrics

Decision threshold $\theta = 1.0$ (§V-C). A payload is a predicted positive iff the firewall does not forward (i.e. reject or escalate). Metrics as defined in §VII-D; confidence intervals are Wilson score at 95 % (benchmarks/lib/stats.mjs); ROC AUC by trapezoidal integration over the swept threshold, with a blocking hit treated as a maximal-confidence positive.

D. C.4 Reproduction

```
cd benchmarks && npm install
npm run corpus      # deepset/prompt-injections -> corpus/payloads.
    json (662 rows)
npm run detection   # confusion matrix + Wilson CIs  -> results/
    detection.{json,md}
npm run roc         # ROC/AUC over the real screen()  -> results/roc.
    json
npm run report      # Section VII placeholder table
```

Committed artifacts of the run reported in Table III live in benchmarks/results/deepset-662-local/. Latency (§VII-C) requires a live edge deployment; run benchmarks/latency.k6.js with WORKER_URL and a provisioned bench tenant key.

residuals. Appendix B reduces solely to the mu-PRF assumption ([BBT16]) and the standard EUF-CMA framework — no unverified external facts. Appendix C's corpus facts are CONFIRMED by direct fetch (verified-facts Part 4b) and its metrics are the executed harness output.

XIV. Data and Code Availability

The Phase-5 injection-classifier pipeline is released to support reproduction of the 99.9 % out-of-distribution result: the corpus builder and leakage-control deduplication (build_corpus.py), the frozen Prompt Guard 2 embedding extractor (extract_pg2_embeddings.py), the head-training and single-touch OOD evaluation (train_head.py, phase5a_bis.py, phase5a_ter.py), the production head finalizer (finalize_head.py), and the pre-registration protocol (docs/phase5a_finetune_scoping.md) that fixed the success criteria before any training code. All evaluation datasets are the license-confirmed public corpora enumerated in Appendix C. The deterministic seeds (20260705/06/07) reproduce the exact train / validation / OOD splits.

Repository: \url{https://github.com/REPLACE-WITH-YOUR-REPO%7D % AIO Apex — set final URL

Artifacts are additionally available from the corresponding author (mo@aioapex.com) on request.

XV. References

Consolidated bibliography. Keys are the mnemonic \cite labels used inline throughout the sections; a camera-ready build would renumber these [1]…[n] via BibTeX. Grouped by type for readability; P = primary/verified, C = canonical dated work, S = secondary (used only qualitatively).

A. Platform documentation (P — Cloudflare, accessed 2026-07-04)

- [C-limits] Cloudflare, "Workers — Limits," developers.cloudflare.com/workers/platform/limits.
- [C-pricing] Cloudflare, "Workers — Pricing," developers.cloudflare.com/workers/platform/pricing.
- [C-changelog-2025] Cloudflare, "Higher CPU limits for Workers," changelog, 2025-03-25.
- [C-changelog-2026] Cloudflare, "Increased subrequest limits," changelog, 2026-02-11.
- [C-do-limits] Cloudflare, "Durable Objects — Limits," developers.cloudflare.com/durable-objects/platform/limits.
- [C-wf-limits] Cloudflare, "Workflows — Limits," developers.cloudflare.com/workflows/reference/limits.

B. Cryptography (P/C)

- [RFC2104] H. Krawczyk, M. Bellare, R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, IETF, Feb. 1997.
- [FIPS198-1] NIST, "The Keyed-Hash Message Authentication Code (HMAC)," FIPS PUB 198-1, 2008.
- [BCK96] M. Bellare, R. Canetti, H. Krawczyk, "Keying Hash Functions for Message Authentication," CRYPTO 1996. (See also M. Bellare, "New Proofs for NMAC and HMAC," CRYPTO 2006.)
- [BBT16] M. Bellare, D. J. Bernstein, S. Tessaro, "Hash-Function Based PRFs: AMAC and Its Multi-User Security," EUROCRYPT 2016.
- [Stripe-sig] Stripe, "Verify webhook signatures," docs.stripe.com/webhooks/signature (accessed 2026-07-04).

C. LLM security & prompt injection (P)

- [OWASP-LLM01] OWASP, "LLM01:2025 Prompt Injection," Top 10 for LLM Applications, genai.owasp.org/llmrisk/llm01-prompt-injection.
- [arXiv-2601.07185] "Defenses Against Prompt Attacks Learn Surface Heuristics," arXiv:2601.07185, 2026.

- [arXiv-2410.22284] M. A. Ayub, S. Majumdar, "Embedding-based classifiers for prompt-injection detection," CAMLIS 2024, arXiv:2410.22284.
- [arXiv-2410.21146] "(direct/indirect/stored prompt-injection taxonomy)," arXiv:2410.21146, 2024.

D. Algorithms & statistics (C)

- [AC75] A. V. Aho, M. J. Corasick, "Efficient String Matching: An Aid to Bibliographic Search," Comm. ACM, 18(6):333–340, 1975.
- [Springer-AC] "(Aho–Corasick multi-pattern matching for signature detection)," Springer, doi:10.1007/978-3-031-96093-2_15.
- [DY83] D. Dolev, A. C. Yao, "On the Security of Public Key Protocols," IEEE Trans. Inf. Theory, 29(2):198–208, 1983.
- [Wilson27] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference," J. Amer. Stat. Assoc., 22(158):209–212, 1927.

E. Database (P — PostgreSQL official docs)

- [PG-json] "JSON Types," postgresql.org/docs/current/datatype-json.html.
- [PG-toast] "Database Physical Storage — TOAST," postgresql.org/docs/current/storage-toast.html.
- [PG-wal-conf] "Write Ahead Log — runtime config," postgresql.org/docs/current/runtime-config-wal.html.
- [PG-wal-config] "WAL Configuration," postgresql.org/docs/current/wal-configuration.html.
- [PG-gin] "GIN Indexes," postgresql.org/docs/current/gin.html.
- [PG-partition] "Table Partitioning," postgresql.org/docs/current/ddl-partitioning.html.

F. Compliance (P)

- [PCI-DSS] PCI Security Standards Council, "PCI-DSS v4.0.1," June 2024. Req 10.2/10.3.1/10.3.2/10.5.1; Req 3.3/3.4.1/3.5.1.
- [AICPA-TSC] AICPA, "Trust Services Criteria" (2017, rev. 2022), Common Criteria CC7.1–CC7.4.

G. Datasets (P — accessed 2026-07-04)

- [DS-PI] deepset, "prompt-injections," Hugging Face Datasets (662 rows; 546/116; Apache 2.0).
- [DS-JB] jackhhao, "jailbreak-classification," Hugging Face Datasets (1,306 rows; Apache 2.0).
- [DS-GA] Lakera, "gandalf_ignore_instructions," Hugging Face Datasets; see also arXiv:2311.01011.

H. Secondary (S — qualitative use only)

- [Layered] "(layered prompt-injection detector; <1 ms Layers 1–2, 300–800 ms model layer)," dev.to. Used only for the model-tier latency order-of-magnitude (§V-A/§V-D).

I. To complete before camera-ready

- [cite] Llama Guard (Meta) — model-guardrail reference (§V-A, §VIII).
- [cite] GitHub webhook signing docs — timestamped HMAC scheme (§VIII).